
Data Science Core Skills

Python

September 2026

Table of contents

Data Science Core Skills	2
Getting Python running	3
Values, names and types	10
Collections	18
Making decisions, repeating work	27
Functions and modules	36
NumPy: arrays and indexing	46
NumPy: computing across arrays	55
pandas: the DataFrame	65
pandas: cleaning, grouping, joining	75
Plotting with Matplotlib	86
Work someone else can rerun	96

Data Science Core Skills

The mathematics, statistics and Python for your MSc, one unit at a time

This booklet is the **Python** track of *Data Science Core Skills*, a self-study course for students on an MSc in Data Science. The whole course — all three subjects, with runnable code and revealable solutions — is at uh-pam.github.io/bootcamp.

Eleven lessons that take you from installing Python to writing an analysis somebody else can rerun. No prior programming is assumed.

Work through them in order — each one builds on the last — and type the code rather than reading it. The single most useful habit is to decide what a line will print *before* you run it.

Read this track first. Every lesson in Mathematics and Statistics ends by checking its own claim in a few lines of code, and those checks are the most useful part of each lesson.

On the site, the lessons of all three subjects are arranged into ten units of about six to seven hours each. The line at the top of each lesson says which unit it belongs to.

Every worked example uses one small table describing a fictional previous cohort: their prior background, hours of practice and final scores. **The data are invented.** They describe no real student and no real cohort.

Getting Python running

Start here · Python · P1 Reading: 4 pages With practice: ~ 2 hours Assumes: nothing — start here

Why this lesson is here

You are going to spend more time reading error messages than writing code. That is not a sign of doing it badly; it is what programming is.

So this lesson is only half about installing Python. The other half is learning to read what the machine tells you when something goes wrong — because for the next few weeks you are working alone, and the traceback is the only thing that can tell you what happened.

Before you start

Nothing. This is the first lesson, and it assumes you have never written a line of code.

It does assume you have worked through [Set up Python](#), which takes about an hour and leaves you with a bootcamp folder and JupyterLab.

Quick check. Can you start JupyterLab and see your data folder in its file panel?

 Show the answer

Open a terminal and type, pressing Enter after each line:

```
cd bootcamp
.venv\Scripts\activate          # Windows PowerShell
source .venv/bin/activate        # macOS or Linux, instead of the line above
jupyter lab
```

A browser tab opens, and the panel on the left lists data. If any line fails, the setup page's *When something goes wrong* section covers the usual causes.

The idea

Python is a language for giving a computer instructions. You write instructions in a plain text file: typed characters only, with none of the fonts or formatting of a Word document. A program called the **interpreter** reads that file and carries them out, one line at a time, top to bottom.

That is genuinely all that happens. There is no separate build step to think about and no magic. When something goes wrong, it went wrong on one specific line, and Python will tell you which.

There are two ways to run instructions, and you will use both:

- A **script** is a file ending in `.py`. You run the whole thing at once. Use scripts when a task should run start-to-finish the same way every time.
- A **notebook** is a file ending in `.ipynb`, made of cells you run one at a time, with output appearing underneath. Use notebooks to explore, and to mix explanation with code.

These notes are written as notebooks-in-spirit: every code block you see was actually run, and its output is printed underneath.

The mechanics

1. What setup gave you

Three things, each of which you will lean on in every lesson:

- **Python** itself — the interpreter.
- **An environment**, the `.venv` folder inside `bootcamp`. `venv` stands for *virtual environment*: a private copy of Python with this course's packages in it. **You must activate it each time you open a new terminal**. Forgetting is the single commonest cause of “but I installed that already”.
- **JupyterLab**, which runs in your browser and is where you write both scripts and notebooks.

Do not repeat the setup commands here. Typing `mkdir bootcamp` again from inside `bootcamp` makes a second `bootcamp` inside the first, and code that looks for `data/cohort.csv` stops finding it.

2. Your first program, as a script

In JupyterLab, choose **File** ▢ **New** ▢ **Python File**. An empty editor opens. Type one line:

```
print("Hello from my own machine")
```

Choose **File** ▢ **Save Python File As...**, name it `hello.py`, and save. It appears in the file panel, inside `bootcamp`.

To run it, choose **File** ▢ **New** ▢ **Terminal**. This terminal opens already inside `bootcamp` and uses the same Python as JupyterLab. Type:

```
python hello.py
```

and press Enter. The line of text appears underneath.

3. The same line, in a notebook

Choose **File** ▢ **New** ▢ **Notebook** (pick *Python 3 (ipykernel)* if asked). Type the same `print(...)` line into the empty cell and press **Shift + Enter**. The output appears directly under the cell, and a new empty cell opens below it.

That is the whole difference: a script runs top to bottom in one go; a notebook runs whichever cell you choose, when you choose. Every code block in these notes can be pasted into a notebook cell.

4. Read the traceback

Now make it fail on purpose. Change `hello.py` to this, save it, and run it again:

```
temperature = "22C"  
print(temperature + 5)
```

Python stops and prints a **traceback**. It looks alarming and is not. It is a report with a fixed shape, and you read it *from the bottom*.

```
temperature = "22C"
print(temperature + 5)
```

TypeError: can only concatenate str (not "int") to str

```
-----
TypeError                                Traceback (most recent call last)
Cell In[1], line 2
      1 temperature = "22C"
----> 2 print(temperature + 5)
TypeError: can only concatenate str (not "int") to str

Traceback (most recent call last):
  File "hello.py", line 2, in <module>
    print(temperature + 5)
    ~~~~~^
TypeError: can only concatenate str (not "int") to str
what went wrong — read this first
```

Figure 1: Every traceback has this shape. The last line names what went wrong; the lines above say where. Read the bottom line first, then look at the line of your own code it points to.

The bottom line is the answer: you tried to join a piece of text to a number, and Python will not guess what you meant. The lines above tell you it happened in `hello.py`, on line 2.

Worked example

A traceback, diagnosed and fixed.

Suppose you are asked to add up three measurements and one of them came from a text file, so it is still text.

This example uses `=` to give a value a name, a list (the square brackets), a loop and a conversion, which P2–P4 teach properly. You do not need to be able to write them yet. Text after a `#` is a comment: Python ignores it, and these notes use comments to explain a line. Read it for the *method* — how to go from a traceback to a fix — and come back to the syntax later.

```
measurements = [12.5, 13.1, "12.9"]
total = sum(measurements)
```

TypeError: unsupported operand type(s) for +: 'float' and 'str'

```
-----
TypeError                                Traceback (most recent call last)
Cell In[3], line 2
      1 measurements = [12.5, 13.1, "12.9"]
----> 2 total = sum(measurements)
TypeError: unsupported operand type(s) for +: 'float' and 'str'
```

Work through it in the fixed order.

Bottom line first. `TypeError: unsupported operand type(s) for +: 'float' and 'str'.` Something tried to add a number to a piece of text.

Which line? The traceback points at the `sum(...)` call.

What actually reached it? Look at the data:

```
measurements = [12.5, 13.1, "12.9"]
for value in measurements:          # run the indented lines once per value
    # repr(value) shows the value exactly; type(value).__name__ is its type's
    #   ↳ name
    print(repr(value), type(value).__name__)
```

```
12.5 float
13.1 float
'12.9' str
```

There it is. The third value is `'12.9'` with quotation marks — text that looks like a number. `repr()` shows the quotation marks that `print()` hides, which is why it is the right tool for this question.

The fix, stated deliberately rather than patched:

```
measurements = [12.5, 13.1, "12.9"]
numbers = [float(value) for value in measurements]  # a new list, each value
#   ↳ made a float
print(sum(numbers))
```

```
38.5
```

Notice what we did *not* do: we did not delete the awkward value, and we did not wrap the whole thing in something that swallows the error. We found out what was actually in the data and converted it on purpose. That habit is worth more than any syntax in this lesson.

Try it yourself

Exercise 1 Predict the final line of the traceback, then run it.

```
print(x)
```

Show the solution

`NameError: name 'x' is not defined.`

Python has no idea what `x` is, because nothing has ever assigned a value to that name. This is different from a `TypeError`, which means the name *does* exist but holds something the operation cannot use.

Exercise 2 This one fails before it runs at all. Why is that different?

```
print("Hello
```

💡 Show the solution

`SyntaxError: unterminated string literal.`

A *string literal* is text typed between quotation marks. The closing quotation mark is missing, so Python cannot even parse (read) the file into instructions. A `SyntaxError` is raised *before* anything executes, which is why you can get one from a line far below code that would otherwise have run.

The practical tell: if nothing at all printed, suspect a syntax error. If some output appeared and then it stopped, the program was running and hit a problem partway.

Exercise 3 You run a script and get `ModuleNotFoundError: No module named 'pandas'`, but you installed pandas ten minutes ago. What is the most likely cause?

💡 Show the solution

You are not in the environment you installed into. Check whether your prompt still shows `(.venv)`; if not, activate it again. If this happened in JupyterLab, it was started from a terminal where the environment was not active: stop it with **Control-C** in that terminal, activate, and start it again.

The general shape of this error: Python is looking, and not finding, so either it was never installed *or you are asking a different Python*. Confirm which interpreter is running with the check in the next section.

Check it in Python

Every other lesson ends by checking a claim. This one checks your setup, which is the claim that matters here.

```
import sys
import numpy, pandas, matplotlib, scipy

print("python      ", sys.version.split()[0])
print("interpreter", sys.executable)
for package in (numpy, pandas, matplotlib, scipy):
    # f"... " puts values into text (P2); :<11 pads each name to 11 characters
    print(f"{package.__name__:<11}", package.__version__)
```

```
python      3.11.14
interpreter /home/hyperspace/anaconda3/bin/python
numpy       1.26.4
pandas      2.3.3
matplotlib  3.10.8
scipy       1.17.0
```

Two things to look at. The version should start with **3.12** or **3.11** — these notes were run on 3.11, which is why the output above says so. And the interpreter path — the chain of folders printed after `interpreter` — should point **inside your .venv folder** — if it points somewhere else, you are running the wrong Python, and that explains almost every “but I installed it” problem you will hit.

Keep this snippet. When you ask anyone for help, the answer is more useful if you paste its output with your question.

Common wrong turns

Where this usually goes wrong

Reading the traceback from the top The top is the least useful part. The bottom line names the problem; work upwards only if you need to know how execution got there.

Assuming an error means you cannot program Everyone who writes code sees errors constantly, including people who have done it for thirty years. An error is the program telling you something specific and true. It is information, not a verdict.

Installing the newest Python The obvious download button gives a version newer than the pinned packages support, and `pip install` then fails, usually with a long build error that never mentions the Python version. Check `python --version`, install 3.12, delete `.venv`, and recreate it.

Forgetting to activate the environment Then you are using a different Python with different packages. In a terminal you opened yourself, the prompt tells you: `no (.venv)` at the start means not active. Inside JupyterLab the prompt does not show it, so run the check in 1.6 and look at the interpreter path instead. This causes more confusion in week one than any other single thing.

Typing the prompt character Examples sometimes show `$` or `>` at the start of a line. That is the prompt, not part of the command. Type only what follows it.

Changing several things at once when stuck Then you cannot tell which change helped. Make one change, run it, look at the result.

Pasting a fix you do not understand It may work today and leave you unable to fix it tomorrow. Before you keep any snippet — from a forum, or from an assistant — make sure you can say what each line does.

Check yourself

Answers are at the foot of the page.

1. Which line of a traceback should you read first?
 - a. the first
 - b. the last
 - c. the one naming your file
 - d. all of them, in order
2. What is the practical difference between a `SyntaxError` and a `NameError`?
3. Why does a virtual environment matter, in one sentence?
4. You see `ModuleNotFoundError` for a package you know you installed. What is the first thing to check?
5. A script prints two lines and then stops with an error. Can the problem be a syntax error? Why or why not?

Show the answers

1. **(b) the last.** It names the kind of error (the *exception type*) and the message — what actually went wrong. The lines above tell you where, which only matters once you know what.

2. A `SyntaxError` means Python could not read your file as instructions, so **nothing ran at all**. A `NameError` means the file ran fine until it reached a name that had never been given a value.
3. It keeps this project's packages separate from everything else on your machine, so installing something here cannot break something there.
4. Whether the environment is active — does your prompt show `(.venv)`? Then confirm with `sys.executable` that the running interpreter is the one inside it.
5. **No**. A syntax error is raised before any code executes, so no output would have appeared. Output followed by a stop means the program was running and hit a problem partway through — which is a useful diagnostic in itself.

Where this shows up next

P2 looks at what values actually are, and why `"22C"` and `22` behave so differently. **P11** returns to environments and versions, and to asking for help in a way that gets a useful answer. Every Mathematics and Statistics lesson ends with a short **Check it in Python** block — the setup you have just built is what makes those runnable.

Values, names and types

Unit 1 · Lesson 1 of 4 · Python · P2 Reading: 5 pages With practice: 1.5 hours Assumes: P1

Why this lesson is here

P1 ended with "22C" + 5 refusing to work. That refusal is the subject of this lesson.

Every value in Python has a **type**, and the type decides what you are allowed to do with it. Most confusing errors in a beginner's first month come down to a value not being the type they assumed. Learning to ask *what type is this, actually?* — and to answer it in one line — removes most of them.

Before you start

You should be able to:

- run a script or a notebook cell — P1;
- read the last line of a traceback — P1.

Quick check. Without running it, what does this print?

```
print(3 + 4)
print("3" + "4")
```

💡 Show the answer

7, then 34.

The + symbol does two different jobs depending on what sits either side of it: arithmetic on numbers, joining on text. That is not Python being inconsistent — it is the type deciding the meaning, which is exactly this lesson's subject.

The idea

Writing `x = 5` does not put 5 into a box called `x`. It creates the value 5 and attaches the **name** `x` to it, like a luggage label. The name is not the thing.

That distinction sounds academic until you meet its consequences, so it is worth a picture:

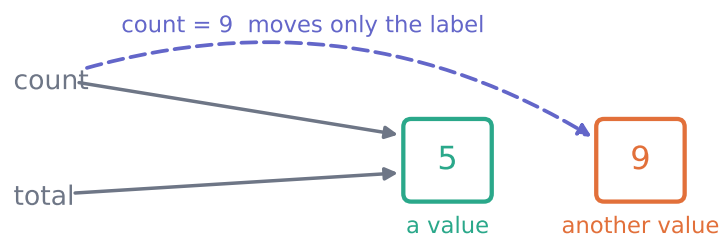


Figure 1: Assignment attaches a name to a value. Two names can label the same value; reassigning one moves that label without touching the value or the other name.

In code:

```
count = 5
total = count      # a second label on the same value
count = 9          # move the first label to a different value

print("count:", count)
print("total:", total)
```

```
count: 9
total: 5
```

`total` still refers to 5. Reassigning `count` moved one label; it did not reach inside `total` and change anything. For the simple values in this lesson that is easy to accept, and in **P3** it becomes something you must think about.

The mechanics

The five types to recognise

```
values = [42, 3.14, True, "hello", None]
for value in values:
    print(f"{repr(value):<8} {type(value).__name__}") # :<8 pads to 8
    ↳ characters
```

```
42      int
3.14    float
True    bool
'hello' str
None    NoneType
```

- **int** — a whole number, of any size. Python will happily hold a 200-digit integer.
- **float** — a number with a decimal part, stored to finite precision. More on that below, because it will bite you.
- **bool** — `True` or `False`. Use these for logical state, not as 1 and 0.
- **str** — text, in quotation marks. `"5"` is text; `5` is a number.
- **None** — the deliberate absence of a value. Not zero, not an empty string; a positive statement that there is nothing here. Its type is called `NoneType`, and `None` is its only value.

Converting on purpose

```
reading = "22.5"
print(float(reading) + 5)
```

```
27.5
```

Conversion works when the value has a sensible reading in the target type, and **raises** an error — stops with a traceback — when it does not:

```
float("22C")
```

```
ValueError: could not convert string to float: '22C'
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[5], line 1
----> 1 float("22C")
ValueError: could not convert string to float: '22C'
```

That failure is a feature. Python could have guessed 22 and thrown away the C, and then your temperature data would silently lose its units.

Floats are approximate

This is the one that surprises everyone:

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)    # == asks "are these equal?"; a single = attaches a
    ~ name
```

```
0.30000000000000004
False
```

Nothing is broken. Computers store numbers in binary, using only the digits 0 and 1, and 0.1 has no exact binary representation — just as 1/3 has no exact decimal one. The tiny error is real and unavoidable.

The consequence for you is a rule: **never test computed floats with ==**. Test whether they are close enough:

```
import numpy as np
print(np.isclose(0.1 + 0.2, 0.3))
```

```
True
```

Integers are exact, so == is fine for them. It is floats that need care, and this is why **M4** checks a derivative with `np.isclose` rather than ==.

f-strings

To build text that contains values, put an `f` before the quotation mark and the values in braces:

```
name = "Life sciences"
mean = 60.5137
count = 57

print(f"{name}: mean {mean:.2f} over {count} students")
```

```
Life sciences: mean 60.51 over 57 students
```

`:.2f` means “as a float, to two decimal places”. This is how every number in these notes is printed, and it saves you from reporting a mean to twelve digits you cannot justify.

Naming

A name should say what the value is, and what unit it is in:

```
temperature_c = 21.5
elapsed_seconds = 90
proportion_passed = 0.83

# :.0% shows a proportion as a percentage with no decimal places
print(f"{temperature_c}°C, {elapsed_seconds}s, {proportion_passed:.0%} passed")
```

21.5°C, 90s, 83% passed

`t = 21.5` is shorter and tells a reader — including you in a fortnight — nothing. Units in the name have prevented more errors than any other habit in this course.

Worked example

Reading a value whose type is not what you assumed.

The cohort file is text on disk. Every column arrives as characters, and something has to decide what they mean. Watch what happens when that decision is skipped:

```
score_from_file = "68.4"
bonus = 5

print("what we have:", repr(score_from_file), type(score_from_file).__name__)
```

what we have: '68.4' str

```
score_from_file + bonus
```

TypeError: can only concatenate str (not "int") to str

```
-----
TypeError                                Traceback (most recent call last)
Cell In[11], line 1
----> 1 score_from_file + bonus
TypeError: can only concatenate str (not "int") to str
```

The error is exactly the one from [P1](#). Now convert on purpose, and check:

```
score = float(score_from_file)
print("converted:", repr(score), type(score).__name__)
print("with bonus:", score + bonus)
```

```
converted: 68.4 float
with bonus: 73.4
```

One thing worth noticing before you move on. "68.4" + "5" would *not* have raised — it would have produced "68.45", silently, and everything downstream would have been wrong without a single error message.

```
print(repr(score_from_file + "5"))
```

```
'68.45'
```

An error that stops you is a good outcome. The dangerous case is the operation that succeeds and means something else.

Try it yourself

Exercise 1 Predict each result, then run them.

```
print(7 / 2)
print(7 // 2)
print(type(7 / 2).__name__)
```

Show the solution

3.5, 3, float.

/ always produces a float, even when the division is exact — 4 / 2 gives 2.0, not 2. // is floor division: it divides and rounds down to a whole number.

The third line is the one people get wrong. 7 / 2 is a float because / always is.

Exercise 2 Why does the first of these print False and the second True?

```
print(0.1 + 0.2 == 0.3)
print(1 + 2 == 3)
```

Show the solution

The second uses integers, which are stored exactly, so the comparison is exact.

The first uses floats. 0.1 and 0.2 cannot be represented exactly in binary, so their sum lands a hair away from 0.3 — about 5.6×10^{-17} above it. == asks whether they are *identical*, and they are not.

Use np.isclose for computed floats. Reserve == for integers, strings, and booleans.

Exercise 3 A colleague stores “no reading was taken” as the number -999. Give two distinct reasons None would be better, and say when -999 might still be forced on you.

💡 Show the solution

Two reasons:

- -999 participates in arithmetic. Take a mean and it silently drags the answer down; nothing warns you. None cannot be added to anything, so the mistake becomes an error you can see.
- -999 is indistinguishable from a genuine measurement of -999. If the quantity could ever legitimately take that value, the two meanings are now merged and cannot be separated.

When it is forced on you: a legacy file format or an instrument that emits a **sentinel** — a stand-in value such as -999 that means “missing”. Then convert it to a proper missing value *as you load the data*, record how many you converted, and never let the sentinel reach a calculation. **P9** does exactly this.

Check it in Python

Two habits, both one line each.

Ask what you actually have rather than assuming:

```
mystery = "12.9"
print(repr(mystery), "→", type(mystery).__name__)
```

```
'12.9' → str
```

`repr()` rather than `print()`, because `print("12.9")` and `print(12.9)` look identical on screen while `repr` shows the quotation marks that distinguish them.

Compare computed floats with a tolerance, and see the difference the choice makes:

```
total = 0.1 + 0.2
print("exact equality :", total == 0.3)
print("np.isclose      :", np.isclose(total, 0.3))
print("the gap         :", total - 0.3)
```

```
exact equality : False
np.isclose      : True
the gap         : 5.551115123125783e-17
```

The gap prints as `5.551115123125783e-17`, where `e-17` is Python’s way of writing $\times 10^{-17}$. So it is about 5.6×10^{-17} — far too small to matter for any measurement, and far too large for `==` to ignore.

`np.isclose` uses a tiny default tolerance. When your data have a natural precision, say so with `atol` (the largest difference you will call equal): exam marks recorded to one decimal place might use `np.isclose(a, b, atol=0.05)`.

Common wrong turns

Where this usually goes wrong

Testing computed floats with == Arithmetic on floats accumulates tiny errors. Use `np.isclose`, and choose a tolerance that means something for your data.

Assuming a number-looking thing is a number "5" is text. It arrives that way from files, forms and web pages. Convert deliberately and check the type when a result surprises you.

Treating None as zero or as an empty string None means *nothing was recorded*. Zero is a measurement. Conflating them changes your answers and hides the fact that data is missing.

Using a sentinel like -999 for missing data It survives into every calculation it touches. If a format forces it on you, convert it the moment you load the file.

Naming values x, df, data, temp You will not remember what they held. `temperature_c` costs eleven characters and carries the unit with it.

Assuming print shows you the type `print` shows a value's *appearance*. "12.9" and 12.9 look the same. Use `repr()` or `type()` when the distinction matters.

Check yourself

Answers are at the foot of the page.

1. What is the type of `7 / 7`?
 - a. `int`
 - b. `float`
 - c. `bool`
 - d. depends on the values
2. `a = [1, 2]`, then `b = a`. Has `b` got its own copy of the list, or another label on the same one?
3. Give a case where `"5" + "5"` is a bug even though nothing raises.
4. Why is `float("22C")` raising an error better than it guessing 22?
5. Someone reports a mean of -142.7 from scores that should lie between 0 and 100. What would you check first?

Show the answers

1. **(b) float.** `/` always produces a float, so `7 / 7` is `1.0`. By contrast, `7 // 7` gives the integer 1.
2. **Another label on the same list.** Nothing was copied. This matters far more for lists than for numbers, because a list can be changed in place — which is P3's subject.
3. Any time the two values are numbers that arrived as text. `"5" + "5"` gives `"55"`, which is not 10 and not an error. Totals that are wildly too large are a classic symptom of adding numbers that were still strings.
4. Because guessing would silently discard information — the C might mean Celsius, or a grade, or a data-entry slip. An error forces you to decide what the value means, once, on purpose.
5. Whether a missing-value sentinel like -999 is in the data and got included in the mean. Count how many values are impossible before doing anything else.

Where this shows up next

P3 puts many values in one container, where the name-versus-value distinction from this lesson starts

to matter a great deal. **P6** adds a fixed type to a whole array at once, which is what makes NumPy fast. **P9** converts a whole column deliberately, and records what would not convert.

Collections

Unit 1 · Lesson 2 of 4 · Python · P3 Reading: 6 pages With practice: 1.5 hours Assumes: P2

Why this lesson is here

Data arrives in quantity. A single score is not interesting; two hundred and forty of them are.

Python gives you four containers for holding many values, and they are not interchangeable. Choosing the wrong one usually still works — which is the problem, because it works slowly, or it lets a duplicate through, or it lets something change underneath you. Choosing well takes one question, asked before you type.

Before you start

You should be able to:

- say what type a value has, and convert it deliberately — P2;
- explain what `a = b` does to names — P2.

Quick check. After `a = 5` and `b = a` and `a = 9`, what is `b`?

 Show the answer

5. Reassigning `a` moved one label; `b` still points at the original value. This lesson is where that idea stops being a curiosity.

The idea

The four containers differ along three questions, and that is the whole of it:

	order	can change?	duplicates
list	ordered	changeable	duplicates ok
tuple	ordered	fixed	duplicates ok
dict	by key	changeable	keys unique
set	unordered	changeable	no duplicates

Figure 1: The four containers, distinguished by three questions. Answer them for your data and the choice is made.

- A **list** is an ordered, changeable sequence. It is the default, and most of the time it is right.
- A **tuple** is an ordered sequence you cannot change. Use it when a group of values belongs together and should stay put.
- A **dictionary** stores values under keys, so you look things up by name rather than by position.
- A **set** holds unique values with no order. Use it to ask *have I seen this before?*

The mechanics

Lists

```
scores = [68.4, 55.1, 72.0, 61.3]

print(scores[0], scores[-1])    # first and last
print(scores[1:3])              # a slice: from 1, up to but not including 3
print(len(scores))
```

```
68.4 61.3
[55.1, 72.0]
4
```

Python counts from **zero**, so the first item is at index 0 and the last is at `len(scores) - 1`. Negative indices count back from the end, which saves you writing `scores[len(scores) - 1]`.

Slicing takes `start:stop`, and `stop` is **not included**. `scores[1:3]` gives two items, not three. That convention looks odd for a day and then becomes convenient, because `scores[:2]` and `scores[2:]` split the list with no overlap and no gap.

Lists change in place:

```
scores.append(59.8)
scores[0] = 70.0
print(scores)
```

```
[70.0, 55.1, 72.0, 61.3, 59.8]
```

The part that catches people

Because a list can change in place, two names on the same list see each other's changes:

```
original = [1, 2, 3]
alias = original          # another label on the SAME list
alias.append(4)

print("original:", original)
print("alias:    ", alias)
```

```
original: [1, 2, 3, 4]
alias:    [1, 2, 3, 4]
```

That is the **P2** picture with real consequences. If you want an independent list, ask for one:

```
original = [1, 2, 3]
independent = original.copy()
independent.append(4)
```

```
print("original:  ", original)
print("independent:", independent)
```

```
original:    [1, 2, 3]
independent: [1, 2, 3, 4]
```

Tuples

Parentheses instead of brackets, and no changing afterwards:

```
point = (52.24, -0.90)      # latitude, longitude
print(point[0])
point[0] = 0
```

```
52.24
```

TypeError: 'tuple' object does not support item assignment

```
-----
TypeError                                Traceback (most recent call last)
Cell In[6], line 3
      1 point = (52.24, -0.90)      # latitude, longitude
      2 print(point[0])
----> 3 point[0] = 0
TypeError: 'tuple' object does not support item assignment
```

The error is the point of a tuple. A coordinate pair, a date, a colour stored as its red, green and blue amounts — things whose parts belong together and should not be edited one at a time.

Dictionaries

```
student = {"id": "S0042", "background": "Life sciences", "score": 68.4}

print(student["background"])
student["completed"] = True      # add a new key
print(list(student.keys()))
```

```
Life sciences
['id', 'background', 'score', 'completed']
```

Look-up is by key, and `student["score"]` says what it means in a way that `row[2]` never will. A key must be a value that cannot change, such as a string or a number. Asking for a key that does not exist raises:

```
student["age"]
```

```
KeyError: 'age'
```

```
-----
KeyError                                Traceback (most recent call last)
Cell In[8], line 1
----> 1 student["age"]
KeyError: 'age'
```

Use `.get()` when absence is expected rather than exceptional:

```
print(student.get("age", "not recorded"))
```

```
not recorded
```

Sets

```
backgrounds = ["Computing", "Life sciences", "Computing", "Social sciences"]
unique = set(backgrounds)

print(unique)
print(len(backgrounds), "values,", len(unique), "distinct")
```

```
{'Life sciences', 'Computing', 'Social sciences'}
4 values, 3 distinct
```

Sets are unordered. The printed order is not sorted and must never be relied on — if you want order, ask for it:

```
print(sorted(unique))
```

```
['Computing', 'Life sciences', 'Social sciences']
```

Membership testing is what sets are really for, and on a large collection it is enormously faster than scanning a list:

```
print("Computing" in unique)
```

```
True
```

Worked example

Summarising the cohort with the right container for each job.

```
import csv                                # load Python's ready-made tools for CSV files
from pathlib import Path                  # and for file paths (imports are explained in
    ~ P5)
```

```
# open the file, read each row, and collect the rows into a list
rows = list(csv.DictReader(Path("data/cohort.csv").open()))
print(len(rows), "rows")
print(rows[0])
```

240 rows

```
{'student_id': 'S0001', 'background': 'Mathematics or Physics', 'diagnostic_score': '89.1', 'h
09-23'}
```

csv.DictReader gives one **dictionary per row**, keyed by column name — so row["final_score"] rather than row[4]. Note that every value is a string: the file is text, and nothing has yet decided what the columns mean.

Now three questions, each answered by a different container.

Which backgrounds appear? Uniqueness matters, order does not — a set:

```
backgrounds = {row["background"] for row in rows if row["background"]}
for name in sorted(backgrounds):
    print(" ", name)
```

```
Computing
Engineering
Life sciences
Mathematics or Physics
Social sciences
```

The curly brackets build a set in one line, keeping the background of each row if it is not empty — an empty string counts as false. This shorthand is a *comprehension*, explained in P4.

What is the mean score in each? Look-up by name — a dictionary of lists:

```
by_background = {}
skipped = 0

for row in rows:
    group, raw = row["background"], row["final_score"]    # two names, two values,
    ↪ in order
    if not group or not raw:    # an empty string counts as false
        skipped += 1           # short for skipped = skipped + 1
        continue               # skip the rest of this pass; go to the next row
    score = float(raw)
    if not 0 <= score <= 100:    # outside 0 to 100 (comparisons: P4)
        skipped += 1
        continue
    by_background.setdefault(group, []).append(score)

for name in sorted(by_background):
    scores = by_background[name]
    print(f" {name:<24} n={len(scores):>3}
    ↪ mean={sum(scores)/len(scores):5.2f}")
print(f"\n {skipped} rows skipped")
```

```

Computing          n= 69  mean=65.85
Engineering        n= 36  mean=67.15
Life sciences      n= 57  mean=60.51
Mathematics or Physics n= 36  mean=75.28
Social sciences    n= 39  mean=60.21

```

```
3 rows skipped
```

`setDefault` creates the empty list the first time a group is seen, so you do not have to check whether the key exists on every row.

Notice the three skipped rows are **counted, not silently dropped**. That number is part of the result; **P9** makes it an audit file.

The five highest scores? Order matters — a sorted list:

```

# every score s from every group's list, gathered into one flat list
all_scores = [s for scores in by_background.values() for s in scores]
print(sorted(all_scores, reverse=True)[:5])

```

```
[91.6, 88.8, 88.5, 86.6, 85.3]
```

Try it yourself

Exercise 1 Predict the output, then run it.

```

values = [10, 20, 30, 40, 50]
print(values[1:4])
print(values[:2])
print(values[-2:])

```

Show the solution

```
[20, 30, 40], [10, 20], [40, 50].
```

The first stops *before* index 4. The second takes everything up to index 2. The third starts two from the end. Together they show why the convention is convenient: `values[:2] + values[2:]` reconstructs the whole list exactly.

Exercise 2 This is meant to leave `baseline` untouched. It does not. Fix it, and say what went wrong.

```

baseline = [1, 2, 3]
adjusted = baseline
adjusted.append(4)

```

Show the solution

`adjusted = baseline` makes a second label on the same list, so `.append` is visible through both names. `baseline` is now `[1, 2, 3, 4]`.

The fix is to ask for an independent list:

```
adjusted = baseline.copy()
```

This is the commonest source of “but I never changed that variable” in a beginner’s code, and it only happens with containers that can change in place — which is why tuples do not have the problem.

Exercise 3 For each, name the container you would use and why in one clause.

- Every distinct postcode in a delivery file, to check whether a new one has been seen before.
- A student’s id, name and score, passed around as one thing.
- Look up a country’s population by its name.
- Daily temperature readings, in the order taken.

💡 Show the solution

- Set** — you want uniqueness, and `in` on a set stays fast however many postcodes there are.
- Tuple** — the three belong together and should not be edited individually. (A dictionary is also defensible if you want the fields named.)
- Dictionary** — look-up by key is exactly what it is for.
- List** — order carries meaning, and you will want to append.

The question to ask each time: *do I look things up by position, by name, or just ask whether something is present?*

Check it in Python

Two checks worth building into the way you work.

Confirm you have an independent copy when you meant to:

```
baseline = [1, 2, 3]
adjusted = baseline.copy()
adjusted.append(4)

# assert does nothing if the test is true; if false, it stops with the message
assert baseline == [1, 2, 3], "baseline was modified"
print("baseline intact:", baseline)
print("same object?    ", adjusted is baseline)
```

```
baseline intact: [1, 2, 3]
same object?    False
```

`is` asks whether two names point at the *same* object; `==` asks whether two objects have the same contents. Two separate lists holding `[1, 2, 3]` are `==` but not `is`.

Count what you dropped, every time:

```

raw = ["68.4", "", "72.1", "not recorded", "55.0"]
usable, dropped = [], []
for value in raw:
    try:
        usable.append(float(value))
    except ValueError:
        dropped.append(value)

print(f"kept {len(usable)} of {len(raw)}; dropped {dropped}")
assert len(usable) + len(dropped) == len(raw)

```

kept 3 of 5; dropped ['', 'not recorded']

try runs the indented line; if it raises a `ValueError` — here, because `float("not recorded")` has no sensible answer — Python jumps to the `except` block instead of stopping, and the loop carries on with the next value. Any *other* error still stops the program, which is what you want.

The assertion says nothing went missing unaccounted for. It is one line and it turns a silent loss into a loud one.

Common wrong turns

Where this usually goes wrong

Assuming assignment copies a container `b = a` gives a second name for the same list. Use `.copy()` when you want an independent one, and remember that numbers and strings never have this problem because they cannot be changed in place.

Relying on the order a set prints in Sets are unordered. The printed order can differ between runs and versions. Call `sorted()` if you need an order.

Using a list for membership tests on large data `x in some_list` scans from the start. `x in some_set` does not. On a hundred thousand items the difference is dramatic.

Indexing rows by number `row[4]` breaks the moment a column is inserted, and it tells a reader nothing. `row["final_score"]` survives, and explains itself.

Being caught by the exclusive slice end `values[1:3]` gives two items. It is a convention worth learning rather than fighting: adjacent slices join up with no overlap.

Dropping rows without counting them Every filtered-out row is a decision. Count them and say how many, or you have no idea whether you removed three or three hundred.

Check yourself

Answers are at the foot of the page.

1. What does `[10, 20, 30, 40][1:3]` give?
 - a. `[10, 20]`
 - b. `[20, 30]`
 - c. `[20, 30, 40]`
 - d. `[10, 20, 30]`
2. `a = [1, 2]; b = a; b.append(3)`. What is `a`, and why?

3. Why can a tuple be a dictionary key when a list cannot?
4. You need to know whether each of a million ids has been seen before. List or set, and why?
5. `print(some_set)` gives a different order on a colleague's machine. Is that a bug?

 Show the answers

1. **(b) [20, 30]**. Start at index 1, stop before index 3.
2. `[1, 2, 3]`. `b` is another label on the same list, and `.append` changes the list itself rather than making a new one.
3. Because a dictionary key must not change after it is stored — the dictionary files it by its contents. A tuple cannot change, so a tuple of strings or numbers is safe; a list can, which would leave the entry unfindable. (Python enforces this: using a list as a key raises `TypeError: unhashable type` — “unhashable” means “cannot be used as a key”. A tuple that contains a list is refused in the same way.)
4. **A set**. `in` on a set does not depend on how many items it holds; on a list it scans, so a million ids means up to a million comparisons each time.
5. **No**. Sets have no order to be wrong about. If order matters to you, that is a signal you should be sorting, or should not be using a set.

Where this shows up next

P4 iterates over these containers and makes decisions about what it finds. **P8** replaces the dictionary-of-lists in the worked example with a labelled table that does the grouping for you. The counted-and-reported skipping in that example becomes the exclusions audit in **P9**.

Making decisions, repeating work

Unit 1 · Lesson 3 of 4 · Python · P4 Reading: 6 pages With practice: 2 hours Assumes: P3

Why this lesson is here

So far your programs have run straight down the page. Two constructions change that, and between them they cover almost everything a program does: **choosing** between alternatives, and **repeating** work over many values.

Everything from cleaning a column to fitting a model is built from these. The skill worth acquiring here is not the syntax, which is small, but the habit of tracing what a loop does *before* you run it.

Before you start

You should be able to:

- index and slice a list, and iterate the idea of “many values” — P3;
- compare values and know what type each is — P2.

Quick check. What does `len([10, 20, 30])` give, and what is the index of its last item?

💡 Show the answer

3, and the last index is 2. Length counts items; indices start at zero, so the last index is always `len(...) - 1`. Off-by-one errors live in that gap.

The idea

A **condition** is a piece of code whose result is `True` or `False`, and it decides which block of code runs:

```
score = 68.4

if score >= 70:
    print("distinction")
elif score >= 50:
    print("pass")
else:
    print("below the pass mark")
```

pass

Two things to notice, because both are Python-specific.

Indentation is syntax. The four spaces are not decoration — they are how Python knows which lines belong to which branch. Get them wrong and the meaning changes, or the file will not parse at all.

elif chains are checked in order, and stop at the first match. A score of 75 satisfies both `>= 70` and `>= 50`, but only the first branch runs. That is why the tests must be written from most to least demanding; reverse them and everything above 50 becomes a pass.

Comparison and logic

```
n = 57
print(n > 50, n == 57, n != 57, 50 <= n <= 100)
print(n > 50 and n < 60)
print(n < 50 or n > 100)
print(not n > 50)
```

True True False True

True

False

False

> means greater than, < less than, >= and <= greater or less than *or equal to*, == equal to, and != not equal to. and needs both sides true, or needs at least one, and not flips the result.

== compares, = assigns. Confusing them is a syntax error in a condition, which is fortunate — in some languages it silently assigns instead.

50 <= n <= 100 chains, and reads exactly as it looks. That is unusual among programming languages and worth using.

The mechanics

for — repeat once per item

```
scores = [68.4, 55.1, 72.0]

for score in scores:
    print(score, "→", "pass" if score >= 50 else "fail")
```

68.4 → pass

55.1 → pass

72.0 → pass

"pass" if score >= 50 else "fail" is the one-line form of if/else: it gives the first value when the condition is true and the second when it is not.

The name score is created fresh on each pass, taking each value in turn. You do not manage an index and you cannot run off the end.

Three helpers you will use constantly:

```
groups = ["Computing", "Life sciences", "Engineering"]
counts = [69, 57, 36]

for position, name in enumerate(groups, start=1):
    print(position, name)
```

```
print()
for name, count in zip(groups, counts):
    print(f"{name:<16} {count}")
```

```
1 Computing
2 Life sciences
3 Engineering
```

```
Computing      69
Life sciences   57
Engineering     36
```

`enumerate` gives you the position alongside the item. `zip` walks two sequences together — and stops at the shorter one, silently, which is worth remembering. The third, `range(a, b)`, counts whole numbers from `a` up to but **not including** `b`, the same convention as slicing: for `i in range(1, 4)` gives `i` the values 1, 2 and 3.

while — repeat until a condition changes

```
remaining = 100.0
halvings = 0

while remaining > 1.0:
    remaining = remaining / 2
    halvings += 1

print(f"{halvings} halvings to get below 1.0, ending at {remaining:.4f}")
```

```
7 halvings to get below 1.0, ending at 0.7812
```

Use `for` when you know what you are iterating over, and `while` when you are waiting for a condition. The danger of `while` is that **something in the loop must change the condition** — if `remaining` never shrank, this would run forever. To stop a runaway loop in a notebook, click the  (stop) button in the toolbar, or choose **Kernel ▾ Interrupt Kernel**; in a terminal, press Control-C.

Tracing a loop before running it

This is the habit that matters. Take a loop and write down what each name holds after each pass:

```
total = 0
for value in [3, 1, 4]:
    total = total + value
print(total)
```

```
8
```

Three lines of paper, and you knew the answer before the machine did. Do this whenever a loop surprises you.

pass	value	total before	total after
1	3	0	3
2	1	3	4
3	4	4	8

Figure 1: Tracing for `value` in `[3, 1, 4]` accumulating a running total. Writing the state after each pass turns a loop from something you hope works into something you have checked.

Comprehensions

A very common loop shape — build a new list by transforming or filtering an old one — has a shorthand:

```
scores = [68.4, 42.0, 72.0, 55.1, 38.5]

passed = [s for s in scores if s >= 50]
rounded = [round(s) for s in scores]

print(passed)
print(rounded)
```

```
[68.4, 72.0, 55.1]
[68, 42, 72, 55, 38]
```

Read `[s for s in scores if s >= 50]` as “s, for each s in scores, where s is at least 50”. It does exactly what the equivalent loop does:

```
passed_long = []
for s in scores:
    if s >= 50:
        passed_long.append(s)

print(passed_long == passed)
```

True

Use a comprehension when it fits on one line and reads as a sentence. When it needs two conditions and a nested loop, write the loop — a comprehension that has to be decoded is worse than the four lines it replaced.

break and continue

```
for value in ["68.4", "", "72.1", "stop", "55.0"]:
    if value == "":
```

```

        continue          # skip this one, carry on
    if value == "stop":
        break             # leave the loop entirely
    print(float(value))

```

68.4

72.1

Worked example

Classifying the cohort, and counting what you could not classify.

```

import csv
from pathlib import Path

rows = list(csv.DictReader(Path("data/cohort.csv").open()))

bands = {"distinction": 0, "merit": 0, "pass": 0, "below": 0}
unusable = []

for row in rows:
    raw = row["final_score"]
    if raw == "":
        unusable.append((row["student_id"], "no score recorded"))
        continue

    score = float(raw)
    if not 0 <= score <= 100:
        unusable.append((row["student_id"], f"impossible score {score}"))
        continue

    if score >= 70:
        bands["distinction"] += 1
    elif score >= 60:
        bands["merit"] += 1
    elif score >= 50:
        bands["pass"] += 1
    else:
        bands["below"] += 1

for name, count in bands.items():
    print(f" {name:<12} {count:>3}")
print(f"\n  classified {sum(bands.values())} of {len(rows)}")
for student, reason in unusable:
    print(f"  excluded {student}: {reason}")

```

```

distinction    69
merit          103
pass           60

```

```

below          6

classified 238 of 240
excluded S0018: no score recorded
excluded S0189: impossible score 187.0

```

Three things in that loop are deliberate and worth copying.

The `continue` cases come **first**, before any arithmetic. A value you cannot use should leave the loop body before it can contaminate anything.

Every exclusion is **recorded with a reason**, not just counted. “Two rows dropped” invites the question “which, and why?”; this answers it in advance.

And the bands are checked from **highest to lowest**, because `elif` stops at the first match. Written the other way round, every score above 50 would land in `pass` and the top two bands would always be zero.

Try it yourself

Exercise 1 Trace this on paper before running it. What does it print?

```

total = 0
for i in range(1, 5):
    total += i
    print(i, total)

```

Show the solution

```

1 1
2 3
3 6
4 10

```

`range(1, 5)` gives 1, 2, 3, 4 — it starts at 1 and stops **before** 5, the same exclusive-end convention as slicing. `total += i` is shorthand for `total = total + i`.

Exercise 2 This is meant to report every score band. Why does it always print `pass` for anything above 50?

```

if score >= 50:
    print("pass")
elif score >= 60:
    print("merit")
elif score >= 70:
    print("distinction")

```

Show the solution

An `elif` chain stops at the **first** true condition. A score of 85 satisfies `>= 50`, so it prints `pass` and the later branches are never reached.

The fix is to order the tests from most to least demanding — ≥ 70 , then ≥ 60 , then ≥ 50 — so the first one that matches is the right one.
 A useful check: if any branch of an `elif` chain can never run, the ordering is wrong.

Exercise 3 Rewrite as a comprehension, then say whether you would keep it. (`name.upper()` gives the name in capital letters.)

```
long_names = []
for name in groups:
    if len(name) > 10:
        long_names.append(name.upper())
```

Show the solution

```
long_names = [name.upper() for name in groups if len(name) > 10]
```

Whether to keep it is a judgement, and the honest answer here is yes: it fits on one line and reads as a sentence — “name uppercased, for each name in groups, where the name is longer than ten characters”.

If it needed a second condition and a nested loop, the four-line version would be clearer. Brevity is not the goal; being read correctly on the first pass is.

Check it in Python

Two checks that catch the two commonest loop bugs.

Did every item get accounted for? A loop that filters should conserve the total:

```
values = [68.4, 42.0, 72.0, 55.1, 38.5]
kept = [v for v in values if v >= 50]
dropped = [v for v in values if v < 50]

assert len(kept) + len(dropped) == len(values)
print(f"kept {len(kept)}, dropped {len(dropped)}, of {len(values)}")
```

kept 3, dropped 2, of 5

If a value could satisfy both conditions or neither, this assertion fires — a > 50 written where ≥ 50 was meant loses every score of exactly 50, and the count catches it whenever the data contains a 50 (these values do not, so try adding one). What it cannot catch is Exercise 2’s misordered `elif` chain: that still puts every score in exactly one branch, just the wrong one. For that you need a case whose answer you already know — check that 85 comes out as `distinction`.

Does the `while` condition actually change? Add a guard while developing:

```
remaining, steps = 100.0, 0

while remaining > 1.0:
```

```
remaining /= 2
steps += 1
assert steps < 1000, "loop is not converging – check the update"

print(f"finished in {steps} steps at {remaining:.4f}")
```

finished in 7 steps at 0.7812

The assertion costs nothing and turns an infinite loop into an immediate, readable failure. Remove it once the logic is settled, or leave it in — a bound you can justify is documentation.

Common wrong turns

Where this usually goes wrong

Ordering an `elif` chain from least to most demanding The first true branch wins, so `>= 50` placed first captures everything. If a branch can never run, the order is wrong.

Forgetting that `range(a, b)` excludes `b` `range(1, 5)` gives four numbers, not five. It is the same convention as slicing, and it makes adjacent ranges join up cleanly.

Writing a `while` whose condition never changes Something in the body must move the loop towards ending. If nothing does, it runs forever. The notebook's  button interrupts it; in a terminal, Control-C.

Modifying a list while looping over it Removing items shifts everything after them, so the loop skips values. Build a new list instead — which is what a comprehension does.

Assuming `zip` complains about unequal lengths It stops at the shorter sequence without a word. If the two should match, assert that they do.

Reaching for a comprehension because it is shorter A comprehension you have to decode is worse than the loop it replaced. Use one when it reads as a sentence.

Check yourself

Answers are at the foot of the page.

1. How many numbers does `range(2, 8)` produce?
 - a. 5
 - b. 6
 - c. 7
 - d. 8
2. What is the difference between `continue` and `break`?
3. `zip([1, 2, 3], ["a", "b"])` — how many pairs, and what happened to the 3?
4. Why must something inside a `while` body change the condition?
5. A classification loop reports zero in your top band every time, though you can see qualifying values in the data. What would you check first?

 Show the answers

1. **(b) 6** — 2, 3, 4, 5, 6, 7. It starts at 2 and stops before 8.
2. `continue` abandons the current pass and moves to the next item; `break` leaves the loop entirely and carries on with the code after it.
3. **Two pairs.** `zip` stops at the shorter sequence, so the 3 is silently dropped. If that would be a bug, assert the lengths match first.
4. Because the condition is re-tested before every pass. If nothing in the body moves it towards `False`, the test never changes and the loop never ends.
5. The order of the `elif` chain. If a less demanding test sits above a more demanding one, the top band is unreachable — every qualifying value is caught by the earlier branch.

Where this shows up next

P5 wraps a loop in a name, so the same logic can be used in several places and tested once. **P7** replaces many of these loops with a single array expression — clearer *and* faster, once the shapes are right. **P9** does this lesson's worked example in three lines, with the exclusions written to a file.

Functions and modules

Unit 1 · Lesson 4 of 4 · Python · P5 Reading: 6 pages With practice: 2 hours Assumes: P4

Why this lesson is here

A function gives a piece of work a **name**, so you can use it in several places and fix it in one.

That is the obvious benefit. The one that matters more for data science is that a named piece of work can be **checked**. A calculation buried in the middle of a script can only be verified by reading it; the same calculation in a function can be handed known inputs and asked whether it returns the right answer. Every lesson after this one leans on that.

Before you start

You should be able to:

- write a loop and a conditional, and indent them correctly — P4;
- work with lists and dictionaries — P3.

Quick check. What is `len("hello")`, and what kind of thing is `len`?

💡 Show the answer

5, and `len` is a **function** — you have been using them since P1. `print`, `float`, `sorted` and `range` are all functions. This lesson is about writing your own.

The idea

```
def standardise(value, mean, sd):  
    """Return how many standard deviations `value` sits above the mean."""  
    return (value - mean) / sd
```

```
print(standardise(72.0, 65.3, 8.4))
```

0.7976190476190479

You need only the arithmetic here: subtract, then divide. The standard deviation itself, a measure of spread, arrives in S2.

Four parts:

- `def` starts a definition, followed by the name.
- The names in parentheses are **parameters** — placeholders for values supplied when the function is called.
- The indented text in triple quotes is a **docstring**: what the function does, for a human.
- `return` hands a value back. Without it, a function returns `None`.

The values you pass in at the call site are **arguments**. Parameters live in the definition; arguments arrive at the call. Keeping the two words apart makes error messages much easier to read.

What is visible where

Names created inside a function are private to it:

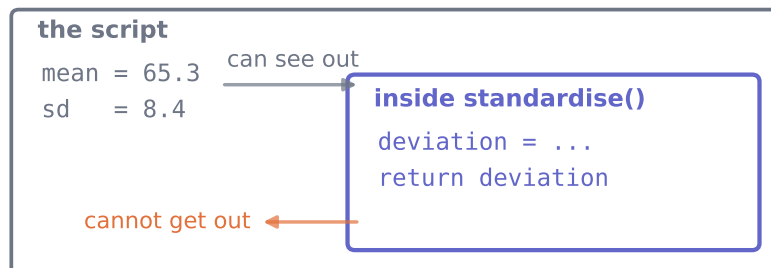


Figure 1: A function can see the names around it, but names it creates are its own. `deviation` does not exist once the function returns.

```
def standardise(value, mean, sd):
    deviation = value - mean
    return deviation / sd

print(standardise(72.0, 65.3, 8.4))
print(deviation)
```

```
0.7976190476190479
```

```
NameError: name 'deviation' is not defined
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[3], line 7
      3     return deviation / sd
      6     print(standardise(72.0, 65.3, 8.4))
----> 7     print(deviation)
NameError: name 'deviation' is not defined
```

`deviation` was created inside and does not exist outside. That is a feature: a function cannot quietly leave things lying around, so you can use one without knowing how it works inside.

The mechanics

Defaults and keyword arguments

```
def summarise(values, decimals=2, label="sample"):
    mean = sum(values) / len(values)
    return f"{label}: n={len(values)}, mean={mean:.{decimals}f}"
```

```
scores = [68.4, 55.1, 72.0, 61.3]
print(summarise(scores))
print(summarise(scores, decimals=4))
print(summarise(scores, label="Life sciences"))
```

```
sample: n=4, mean=64.20
sample: n=4, mean=64.2000
Life sciences: n=4, mean=64.20
```

Parameters with defaults come after those without. At the call site, naming the argument — `decimals=4` rather than a bare `4` — makes the line readable and means you can supply them in any order.

! Never use a list or dictionary as a default value. It is created once, when the function is defined, and then shared by every call:

```
def add_reading(value, readings=[]):      # do not do this
    readings.append(value)
    return readings

print(add_reading(1))
print(add_reading(2))      # the list from the first call is still there
```

```
[1]
[1, 2]
```

Use `None` as the default and create the list inside:

```
def add_reading(value, readings=None):
    if readings is None:
        readings = []
    readings.append(value)
    return readings

print(add_reading(1))
print(add_reading(2))
```

```
[1]
[2]
```

Doing one thing

A function should be describable in one sentence without using “and”. Compare:

```
def mean(values):
    """Return the arithmetic mean of a non-empty sequence."""
```

```

    return sum(values) / len(values)

def usable(values, low=0, high=100):
    """Return only the values inside [low, high]."""
    return [v for v in values if low <= v <= high]

raw = [68.4, 187.0, 55.1, 72.0]
print(mean(usable(raw)))

```

65.16666666666667

Two small functions, each checkable on its own, beat one that filters *and* averages and can only be tested as a lump.

assert — a check you leave in the code

```

def mean(values):
    assert len(values) > 0, "mean of an empty sequence is undefined"
    return sum(values) / len(values)

print(mean([1, 2, 3]))

```

2.0

```
mean([])
```

AssertionError: mean of an empty sequence is undefined

```

-----
AssertionError                                Traceback (most recent call last)
Cell In[9], line 1
----> 1 mean([])
Cell In[8], line 2, in mean(values)
      1 def mean(values):
----> 2     assert len(values) > 0, "mean of an empty sequence is undefined"
      3     return sum(values) / len(values)
AssertionError: mean of an empty sequence is undefined

```

Without the assertion this raises `ZeroDivisionError` from the division — `sum([])` is 0, and so is `len([])` — which tells you what broke but not what you did wrong. With it, the message names the actual mistake.

Modules

A module is a file of Python you can use from another file. Import at the top:

```
import math
import statistics as stats

print(math.sqrt(2))
print(stats.median([68.4, 55.1, 72.0]))
```

```
1.4142135623730951
```

```
68.4
```

`import math` keeps the origin visible — `math.sqrt` says where `sqrt` came from. `import statistics as stats` shortens a long name; `import numpy as np` is the same move, and the abbreviations for `numpy`, `pandas` and `matplotlib` are universal conventions worth following exactly.

i Avoid from module `import *`. It pulls every name in, so you cannot tell what came from where, and a later import can silently replace an earlier name. The cost of typing `math.` is small; the cost of a name collision you cannot see is not.

The **standard library** is the set of modules that come with Python itself, so they need no installing, unlike `numpy` or `pandas`. A short list worth knowing exists: `math`, `statistics`, `pathlib` for file paths, `csv`, `json`, `random`, `datetime`.

Worked example

A checked function, used on the cohort.

```
def summarise_group(scores, low=0, high=100):
    """Summarise a group's scores, reporting what was excluded.

    Returns a dictionary with the count kept, the count excluded, the mean and
    the sample standard deviation. Raises if nothing usable remains.
    """
    usable = [s for s in scores if low <= s <= high]
    excluded = len(scores) - len(usable)

    if len(usable) < 2:
        # raise stops the function with this error, as a failed assert would
        raise ValueError(f"need at least 2 usable scores, got {len(usable)}")

    mean = sum(usable) / len(usable)
    # ** is "to the power of": square each distance from the mean, then add up
    variance = sum((s - mean)**2 for s in usable) / (len(usable) - 1)

    return {"n": len(usable), "excluded": excluded,
            "mean": mean, "sd": variance**0.5}    # **0.5 is the square root
```

Check it against values you can verify by hand *before* pointing it at real data:

```

result = summarise_group([2, 4, 4, 4, 5, 5, 7, 9])
print(result)

assert result["n"] == 8
# abs() drops the sign; 1e-12 is 0.000000000001, so "equal, allowing for
↳ rounding"
assert abs(result["mean"] - 5.0) < 1e-12
assert abs(result["sd"] - 2.13809) < 1e-4          # sample sd, n-1 divisor
assert summarise_group([1, 2, 3, 187])["excluded"] == 1
print("all checks passed")

```

```

{'n': 8, 'excluded': 0, 'mean': 5.0, 'sd': 2.138089935299395}
all checks passed

```

You can do both by hand. The mean of those eight numbers is 5 by inspection. Their distances from 5 are $-3, -1, -1, -1, 0, 0, 2, 4$; the squares add to 32; and $\sqrt{32/7} = 2.13809$, dividing by $n - 1 = 7$ as **S8** requires. Now use it:

```

import csv
from pathlib import Path

rows = list(csv.DictReader(Path("data/cohort.csv").open()))

by_group = {}
for row in rows:
    if row["background"] and row["final_score"]:
        by_group.setdefault(row["background"],
↳ []).append(float(row["final_score"]))

for name in sorted(by_group):
    r = summarise_group(by_group[name])
    note = f" ({r['excluded']} excluded)" if r["excluded"] else ""
    print(f" {name:<24} n={r['n']:>3} mean={r['mean']:5.2f}
↳ sd={r['sd']:4.2f}{note}")

```

Computing	n= 69	mean=65.85	sd=6.60	
Engineering	n= 36	mean=67.15	sd=5.33	
Life sciences	n= 57	mean=60.51	sd=7.29	(1 excluded)
Mathematics or Physics	n= 36	mean=75.28	sd=7.63	
Social sciences	n= 39	mean=60.21	sd=6.58	

The one excluded value is the impossible score of 187 planted in the file. The function found it, reported it, and did not average it in — because the exclusion rule lives in one place and is tested.

Try it yourself

Exercise 1 Write `larger(a, b)` returning the larger of two numbers, then write three assertions that would catch a wrong implementation — including the equal case.

 Show the solution

```
def larger(a, b):
    """Return the larger of two numbers; either if they are equal."""
    return a if a >= b else b

assert larger(3, 7) == 7      # second larger
assert larger(7, 3) == 7      # first larger
assert larger(4, 4) == 4      # equal
```

The equal case is the one people omit, and it is where a version built from two strict tests — if `a > b`: return `a`, then if `b > a`: return `b` — matches neither, falls off the end, and returns `None`. Testing only `larger(3, 7)` would pass a function that always returns its second argument.

Exercise 2 Why does this print `[1, 2]` on the second call rather than `[2]`?

```
def collect(value, into=[]):
    into.append(value)
    return into

print(collect(1))
print(collect(2))
```

 Show the solution

The default `[]` is created **once**, when the `def` line runs — not on each call. Every call that does not supply `into` shares that same list, so values accumulate across calls.

The fix:

```
def collect(value, into=None):
    if into is None:
        into = []
    into.append(value)
    return into
```

The rule: a default value must be **immutable** — impossible to change in place. Numbers, strings, `True`, `None` and tuples are safe; lists, dictionaries and sets are not.

Exercise 3 This function does three things. Split it, and say what you gain.

```
def process(rows):
    scores = [float(r["final_score"]) for r in rows if r["final_score"]]
    valid = [s for s in scores if 0 <= s <= 100]
    return sum(valid) / len(valid)
```

 Show the solution

```
def extract_scores(rows):
    """Pull final_score out of rows that have one, as floats."""
    return [float(r["final_score"]) for r in rows if r["final_score"]]

def usable(scores, low=0, high=100):
    """Keep only scores inside the plausible range."""
    return [s for s in scores if low <= s <= high]

def mean(values):
    assert values, "mean of an empty sequence is undefined" # an empty list
    → counts as false
    return sum(values) / len(values)
```

Any split into these three jobs — extract, filter, average — answers the exercise; your names may differ. The `low` and `high` parameters and the assertion are improvements, not requirements. A good answer to “what you gain” mentions testing each piece separately.

What you gain: each piece can be tested with two lines and no CSV file; the plausible range becomes a visible, changeable parameter rather than a hard-coded 0 and 100 buried three lines deep; and when a mean comes out wrong you can find out *which* stage was responsible.

What you also gain, less obviously: `usable` now has a name, so the fact that filtering happened is visible at the call site instead of hidden.

Check it in Python

Write the checks **before** the function is used in anger, on inputs whose answer you already know.

```
def sample_sd(values):
    """Sample standard deviation, with the n-1 divisor."""
    if len(values) < 2:
        raise ValueError("need at least two values")
    mean = sum(values) / len(values)
    return (sum((v - mean)**2 for v in values) / (len(values) - 1))**0.5

import numpy as np
# np.std is numpy's standard deviation; ddof=1 makes it divide by n-1, as ours
→ does
for case in ([2, 4, 4, 4, 5, 5, 7, 9], [1.5, 2.5], [10, 10, 10]):
    mine, theirs = sample_sd(case), np.std(case, ddof=1)
    print(f"{str(case):<28} {mine:8.5f} vs numpy {theirs:8.5f} "
          f"agree: {np.isclose(mine, theirs)}")
```

[2, 4, 4, 4, 5, 5, 7, 9]	2.13809	vs numpy	2.13809	agree: True
[1.5, 2.5]	0.70711	vs numpy	0.70711	agree: True
[10, 10, 10]	0.00000	vs numpy	0.00000	agree: True

Comparing against an independent implementation is the strongest cheap check there is. Note the

three cases: an ordinary one, the smallest legal input, and one with no variation at all — ordinary, edge, and degenerate.

And check that it refuses what it should:

```
try:
    sample_sd([5])
except ValueError as error:
    print("correctly refused:", error)
```

correctly refused: need at least two values

A function that fails loudly on bad input is more useful than one that returns a number you cannot trust.

Common wrong turns

Where this usually goes wrong

A mutable default argument `def f(x, acc=[])` creates that list once and shares it across all calls. Use `None` and create it inside.

Printing instead of returning A function that prints gives you nothing to test or reuse. Return the value and let the caller decide whether to print it.

One function doing several jobs If describing it needs the word “and”, it is two functions. Small pieces are testable pieces.

Testing only the easy case `larger(3, 7)` passes for a function that always returns its second argument. Test ordinary, edge and degenerate inputs.

from module import * Hides where names came from and lets a later import silently replace an earlier one. Import the module, or import specific names.

Assuming a function can change the caller’s variables Assigning to a name inside a function creates a local name; it never changes what the caller’s name refers to. Results come back through the return value — which is why a function with no return gives you `None`. The exception is a list the function *changes in place*: `readings.append(value)` alters the one list both names label, as the default-argument trap above shows.

Check yourself

Answers are at the foot of the page.

1. What does a function return if it has no return statement?
 - a. 0
 - b. None
 - c. the last value computed
 - d. it is an error
2. What is the difference between a parameter and an argument?
3. Why is `def f(x, acc=[])` dangerous?

4. Give one reason to prefer returning a value over printing it.
5. You test `larger(a, b)` only with `larger(2, 9)` and it passes. Name an incorrect implementation that would also pass.

 Show the answers

1. **(b) None.** Which is why a function that prints but does not return gives you `None` if you try to use its result.
2. A **parameter** is the name in the definition; an **argument** is the value supplied at the call. `def f(x)` declares a parameter; `f(3)` passes an argument.
3. The default list is created once at definition time and shared by every call that does not supply one, so values accumulate between unrelated calls.
4. Any of: the value can be tested; it can be reused in a larger calculation; the caller chooses the formatting; and the function stays usable in a script that should print nothing at all.
5. `def larger(a, b): return b`—it returns 9, which is correct for that one case. So would `return max(a, b) if a < b else b`. Testing the reversed order and the equal case rules both out.

Where this shows up next

P6 starts using functions from NumPy in earnest, and the import conventions from this lesson become universal. **P11** moves functions into their own `.py` files and runs the checks with `pytest` instead of bare assertions. Every **Check it in Python** block in the Mathematics and Statistics tracks is this lesson's habit applied to a mathematical claim.

NumPy: arrays and indexing

Unit 2 · Lesson 1 of 3 · Python · P6 Reading: 7 pages With practice: 2 hours Assumes: P5

Why this lesson is here

A Python list can hold anything, in any mix, and grow at either end. That flexibility costs speed and, more importantly here, costs *meaning* — a list gives you no way to say “these 240 numbers are one variable”.

A NumPy **array** trades the flexibility away. Every element has the same type, the shape is fixed, and the whole thing sits in one block of memory. In exchange you get arithmetic on entire columns at once, and a vocabulary — shape, axis, dtype — that the rest of data science is written in.

Before you start

You should be able to:

- index and slice a list — P3;
- import a module under an alias — P5;
- say what type a value has — P2.

Quick check. What does `[10, 20, 30][1:3]` give?

💡 Show the answer

`[20, 30]`. Start at index 1, stop *before* index 3. Array slicing uses exactly the same convention, so this transfers directly.

The idea

```
import numpy as np

scores = np.array([68.4, 55.1, 72.0, 61.3])
print(scores)
print("shape", scores.shape, " ndim", scores.ndim, " dtype", scores.dtype)
```

```
[68.4 55.1 72.  61.3]
shape (4,)  ndim 1  dtype float64
```

Three **attributes** — values stored on the array, read with a dot and no brackets — describe every array, and you should be able to state all three before doing arithmetic:

- **shape** — how many elements along each dimension, as a tuple.
- **ndim** — how many dimensions. `len(shape)`.
- **dtype** — the single type every element has.

shape is `(4,)` with a trailing comma because it is a one-element tuple: four values along one dimension. `(4,)` and `(4, 1)` are different things, and P7 is where that difference bites.

The immediate payoff is that arithmetic applies to everything at once:

```
print(scores + 5)
print(scores > 60)
```

```
[73.4 60.1 77. 66.3]
[ True False  True  True]
```

A list will not do either. `[1, 2] + 5` raises, and `[1, 2] > 1` is meaningless.

The mechanics

Making arrays

```
print(np.zeros(4))
print(np.ones((2, 3)))
print(np.arange(0, 10, 2))
print(np.linspace(0, 1, 5))
```

```
[0. 0. 0. 0.]
[[1. 1. 1.]
 [1. 1. 1.]]
[0 2 4 6 8]
[0.   0.25 0.5  0.75 1.  ]
```

`arange(start, stop, step)` follows the usual exclusive-stop convention, so it never reaches `stop`. `linspace(start, stop, n)` gives exactly `n` points and **does** include both ends — a deliberate difference, and the reason `linspace` is the right choice for plotting a curve.

dtype is part of the calculation

```
whole = np.array([1, 2, 3])
print(whole.dtype)

whole[0] = 2.7          # assigning a float into an integer array
print(whole)
```

```
int64
[2 2 3]
```

The 2.7 was truncated to 2 without a word of warning. `dtype` is not a display detail — it changes results. If a column will hold fractions, make it float from the start:

```
decimal = np.array([1, 2, 3], dtype=float)
decimal[0] = 2.7
print(decimal, decimal.dtype)
```

```
[2.7 2.  3. ] float64
```

Mixing types in the input forces a common dtype, which is rarely what you want:

```
print(np.array([1, 2.5, 3]).dtype)
print(np.array([1, 2, "3"]).dtype)
```

```
float64
<U21
```

That second one turned your numbers into text. Inspect the dtype whenever an array comes from somewhere you did not control.

Two dimensions

```
table = np.arange(12).reshape(3, 4)
print(table)
print("shape", table.shape)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
shape (3, 4)
```

Index with [row, column]:

```
print(table[0, 0])    # first row, first column
print(table[2, -1])   # last row, last column
print(table[1])        # a whole row
print(table[:, 1])     # a whole column
```

```
0
11
[4 5 6 7]
[1 5 9]
```

`table[:, 1]` reads as “every row, column 1”. The colon means *all of this dimension*, and it is how you take a column out of a table.

Boolean indexing

The single most useful thing in this lesson:

```
scores = np.array([68.4, 42.0, 72.0, 55.1, 187.0])

mask = (scores >= 50) & (scores <= 100)
print(mask)
print(scores[mask])
```

```
[ True False  True  True False]
[68.4 72.  55.1]
```

The comparison produces an array of True/False, and indexing with it keeps only the True positions. Two rules:

- Use & and |, not and and or. The word forms ask for one answer about the whole array; the symbols work element by element.
- **Parenthesise each comparison.** `scores >= 50 & scores <= 100` binds in the wrong order and fails confusingly.

Counting is then free, because True sums as 1:

```
print(f"{mask.sum()} of {mask.size} usable")
```

```
3 of 5 usable
```

Views and copies

This is the concept that catches people, and it is worth slowing down for.

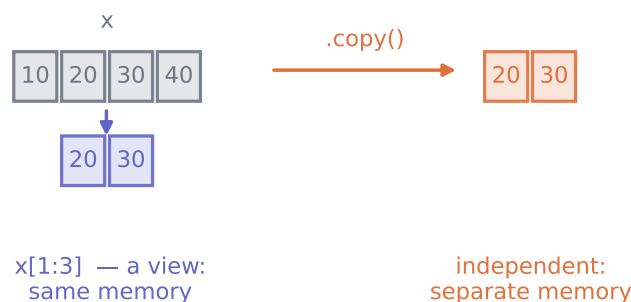


Figure 1: A basic slice is a *view* — a second way of looking at the same memory, so writing through it changes the original. Advanced indexing, and `.copy()`, produce independent data.

```
x = np.array([10, 20, 30, 40])
window = x[1:3]          # a view
window[0] = 99

print("x      ", x)
print("window", window)
```

```
x      [10 99 30 40]
window [99 30]
```

Changing the view changed `x`. Nothing was copied — `window` is a second way of looking at the same block of memory. That is deliberate: slicing a large array would be expensive if it duplicated the data.

When you want independence, say so:

```
x = np.array([10, 20, 30, 40])
window = x[1:3].copy()
window[0] = 99

print("x      ", x)
print("window", window)
```

```
x      [10 20 30 40]
window [99 30]
```

You can always ask:

```
x = np.array([10, 20, 30, 40])
print("slice shares memory: ", np.shares_memory(x, x[1:3]))
print("boolean shares memory:", np.shares_memory(x, x[x > 15]))
```

```
slice shares memory:  True
boolean shares memory: False
```

Basic slicing gives a view; boolean indexing and **fancy** (or *advanced*) indexing — picking positions with a list, as in `x[[0, 2]]` — give a copy. When in doubt, `np.shares_memory` answers it in one line.

Worked example

Loading the cohort’s numeric columns and finding the usable rows.

```
import numpy as np

raw = np.genfromtxt("data/cohort.csv", delimiter=",", skip_header=1,
                    usecols=(2, 3, 4))          # diagnostic, hours, final
print("shape", raw.shape, " dtype", raw.dtype)
print(raw[:3])
```

```
shape (240, 3)  dtype float64
[[89.1 12.9 83.6]
 [42.4 22.8 56.6]
 [68.4  1.7 58.5]]
```

`genfromtxt` turned anything unparseable into `nan` — “not a number”, the float value meaning *this is missing*. It is there in the data:

```
missing = np.isnan(row).any(axis=1) # for each row: is any value in it nan?
print(f"rows with a missing value: {missing.sum()}")
print("row indices:", np.flatnonzero(missing))
```

```
rows with a missing value: 1
row indices: [17]
```

Now build the usable mask, one condition at a time so each is visible:

```
complete = ~np.isnan(row).any(axis=1)
in_range = (row[:, 2] >= 0) & (row[:, 2] <= 100)
usable = complete & in_range

print(f"complete rows : {complete.sum()}")
print(f"score in range: {in_range.sum()}")
print(f"both           : {usable.sum()} of {row.shape[0]}")

data = row[usable]
print("kept shape", data.shape)
```

```
complete rows : 239
score in range: 238
both           : 238 of 240
kept shape (238, 3)
```

~ negates a boolean array — read it as “not”. And note `in_range` is computed on the `row` array, where every comparison with `nan` except `!=` gives `False`, so it already excludes the missing row; combining both conditions explicitly is clearer than relying on that.

Finally, a sanity check on the result rather than a hope:

```
final = data[:, 2]
print(f"n      {final.size}")
print(f"min    {final.min():.1f}")
print(f"max     {final.max():.1f}")
print(f"mean    {final.mean():.2f}")
# .any(): at least one element is True; .all(): every element is True
assert not np.isnan(data).any(), "a missing value survived the filter"
assert (final >= 0).all() and (final <= 100).all(), "an impossible score
↳ survived"
print("filters did what they claimed")
```

```
n      238
min    44.5
max    91.6
mean   65.27
filters did what they claimed
```

Two assertions, two lines, and the class of error where a filter silently fails to filter is now impossible to miss.

Try it yourself

Exercise 1 Predict shape, ndim and dtype for each, then check.

```
a = np.array([1, 2, 3])
b = np.array([[1.0, 2.0], [3.0, 4.0]])
c = np.zeros((2, 3, 4))
```

💡 Show the solution

- a — shape (3,), ndim 1, dtype int64 (may be int32 on Windows).
- b — shape (2, 2), ndim 2, dtype float64, because the literals are written with decimal points.
- c — shape (2, 3, 4), ndim 3, dtype float64. zeros gives floats unless told otherwise.

The dtype of a is the one worth noting: writing 1 rather than 1.0 gives you an integer array, and assigning 2.7 into it later would silently truncate.

Exercise 2 Why does the first change x and the second not?

```
x = np.array([1, 2, 3, 4])
x[:2][0] = 99          # first

y = np.array([1, 2, 3, 4])
y[y < 3][0] = 99      # second
```

💡 Show the solution

`x[:2]` is basic slicing, so it returns a **view** into x's memory. Writing through it writes into x. `y[y < 3]` is boolean indexing, which must gather scattered elements, so it returns a **copy**. The 99 is written into that temporary copy, which is then discarded — y is untouched and no error is raised.

The second is the more dangerous of the two, because it silently does nothing. To assign through a boolean mask, put the mask on the left of the assignment:

```
y[y < 3] = 99
```

Exercise 3 Write one expression giving the diagnostic scores of rows whose final score is at least 70, from the `raw` array above. Then say how many there are.

💡 Show the solution

```
high = raw[(raw[:, 2] >= 70) & ~np.isnan(raw[:, 2]), 0]
print(high.size)
```

Reading it outward: `raw[:, 2]` is the final-score column; the comparison makes a boolean mask; `& ~np.isnan(...)` drops the missing row explicitly; and the trailing `, 0` selects the diagnostic column from the rows that survived.

Strictly the `~np.isnan` is redundant, because `nan >= 70` is already `False` — but relying on that is the kind of cleverness that fails when someone flips the mask to find the *other* rows. `~(raw[:,`

2] >= 70) is True for the missing row, so it would quietly count as a low score. Say what you mean.

Check it in Python

Ask whether you are holding a view before writing into anything:

```
x = np.array([1.0, 2.0, 3.0, 4.0])
piece = x[1:3]
print("shares memory with x:", np.shares_memory(x, piece))
```

shares memory with x: True

Assert the shape you expect, so a mistake stops the program rather than propagating:

```
table = np.arange(12).reshape(3, 4)
column = table[:, 1]

assert column.shape == (3,), f"expected (3,), got {column.shape}"
print("column", column, "shape", column.shape)
```

column [1 5 9] shape (3,)

Check that a filter filtered. The failure mode is not an error; it is a filter that quietly kept something:

```
values = np.array([68.4, np.nan, 187.0, 55.1])
kept = values[~np.isnan(values) & (values <= 100)]

print("kept:", kept)
assert not np.isnan(kept).any()
assert (kept <= 100).all()
print(f"{kept.size} of {values.size} kept, and both conditions verified")
```

kept: [68.4 55.1]
2 of 4 kept, and both conditions verified

Common wrong turns

Where this usually goes wrong

Writing through a boolean index and expecting it to stick `y[y < 3][0] = 99` modifies a temporary copy and is silently discarded. Put the mask on the left: `y[y < 3] = 99`.

Forgetting a slice is a view `piece = x[1:3]` then `piece[0] = 99` changes `x`. Use `.copy()` when you want independence, and `np.shares_memory` when you are unsure.

Using and / or on arrays They ask for a single truth value about the whole array and raise. Use `&` and `|`, with each comparison in its own parentheses.

Ignoring dtype An integer array truncates anything fractional assigned into it, without warning. Create it as float if it will hold fractions.

Assuming a mixed list gives you numbers `np.array([1, 2, "3"])` produces an array of *strings*. Check the dtype when the source is not under your control.

Comparing against nan with == `nan == nan` is False, and so are `<`, `>`, `<=` and `>=`; only `nan != nan` is True. Use `np.isnan`.

Check yourself

Answers are at the foot of the page.

1. What is the shape of `np.zeros((3, 4))[:, 2]`?
 - a. (4,)
 - b. (3,)
 - c. (3, 1)
 - d. (1, 3)
2. `x = np.array([1,2,3]); y = x[:2]; y[0] = 9`. What is `x`, and why?
3. Why must each comparison be parenthesised in `(a > 1) & (a < 5)`?
4. `np.array([1, 2, 3])[0] = 2.9` — what does the array hold afterwards?
5. A colleague filters missing values with `values[values != np.nan]` and gets everything back. What went wrong?

Show the answers

1. **(b) (3,)**. Taking one column from a (3, 4) array leaves one value per row, and the column dimension disappears.
2. [9, 2, 3]. `x[:2]` is a view into the same memory, so writing through `y` writes into `x`.
3. Because `&` binds more tightly than `>` and `<`. Without parentheses, `a > 1 & a < 5` is read as `a > (1 & a) < 5`, which is not what you meant and usually raises.
4. [2, 2, 3]. The array is integer dtype, so 2.9 is truncated to 2 — no error, no warning.
5. `nan` is not equal to anything, including itself, so `values != np.nan` is True everywhere and nothing is filtered. Use `~np.isnan(values)`.

Where this shows up next

P7 does arithmetic *across* arrays — broadcasting, reductions by axis, and the shape traps that come with them. **P8** puts names on the columns so you stop writing `raw[:, 2]` and start writing `df["final_score"]`. **M8** treats a row of this table as a vector, and gives the arithmetic a geometric meaning.

NumPy: computing across arrays

Unit 2 · Lesson 2 of 3 · Python · P7 Reading: 7 pages With practice: 2 hours Assumes: P6

Why this lesson is here

Data science is arithmetic done to whole tables at once. Three ideas make that possible: operations that apply to every element, *broadcasting* that lets differently-shaped arrays combine, and *reductions* that collapse an array along one direction.

They are also the three places where beginners most often get a wrong answer rather than an error message. NumPy will happily subtract the wrong thing from the wrong thing and hand you a perfectly shaped result. This lesson is as much about noticing that as about the syntax.

Before you start

You should be able to:

- create an array and inspect its shape, ndim and dtype — P6;
- index and slice, and say whether you are holding a view or a copy — P6;
- write a for loop over a list — P4.

Quick check. What are the shape and ndim of `np.zeros((3, 4))`?

💡 Show the answer

shape is (3, 4) and ndim is 2 — three rows, four columns, two dimensions. If that was not immediate, reread P6 first; every idea below is expressed in terms of shape.

The idea

In plain Python, doing something to every number means writing a loop:

```
scores = [58.0, 71.5, 64.0, 80.5]

adjusted = []
for score in scores:
    adjusted.append(score * 1.1)
print(adjusted)
```

```
[63.800000000000004, 78.65, 70.4, 88.55000000000001]
```

In NumPy you write the operation once and it applies to everything:

```
import numpy as np

scores = np.array([58.0, 71.5, 64.0, 80.5])
print(scores * 1.1)
```

```
[63.8  78.65 70.4  88.55]
```

This is not only shorter. The loop still happens — it happens inside **compiled** code (already translated into machine instructions before you run it) rather than in the Python interpreter, which reads your code step by step. For large arrays that is dramatically faster. But speed is the lesser reason. The real gain is that `scores * 1.1` says *what* you want, while the loop says *how* to get it, and the first is much easier to read and much harder to get subtly wrong.

The same applies to comparisons, which produce arrays of True/False:

```
print(scores > 65)
print(scores[scores > 65])
```

```
[False  True False  True]
[71.5  80.5]
```

The mechanics

Broadcasting

Two arrays of the same shape combine element by element. Arrays of *different* shapes may still combine, if their shapes are compatible. NumPy compares them **from the right**, and two dimensions are compatible when they are equal or one of them is 1.

```
table = np.arange(6).reshape(2, 3)      # shape (2, 3)
offset = np.array([10, 20, 30])         # shape (3,)

print(table)
print(table + offset)
```

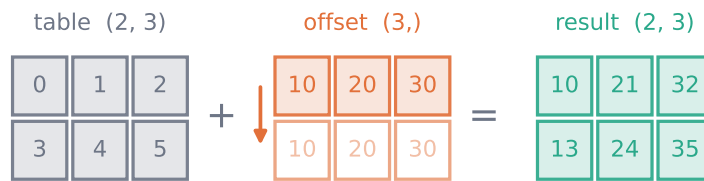
```
[[0 1 2]
 [3 4 5]]
[[10 21 32]
 [13 24 35]]
```

The (3,) array was stretched across both rows. Nothing was copied in memory — NumPy reuses the same values — but the *result* is as if it had been.

This is concise and it is also where silent errors live. A one-dimensional array of length 3 has shape (3,). The same numbers as a column have shape (3, 1). They do not behave the same way:

```
v = np.array([10.0, 20.0, 30.0])

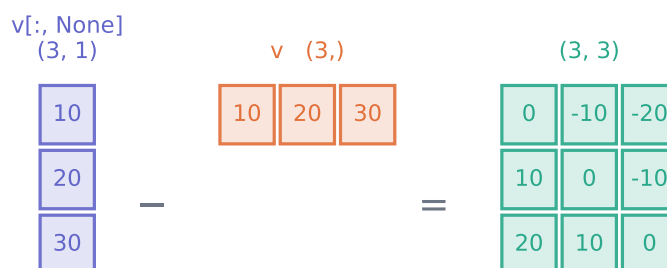
print("v.shape      ", v.shape)
# None inside square brackets inserts a new dimension of length 1 at that place
print("v[:, None].shape ", v[:, None].shape)
print("(v - v).shape    ", (v - v).shape)
print("(v[:, None] - v).shape ", (v[:, None] - v).shape, " <-- 3x3, not 3")
```



the second row is stretched, not stored

Figure 1: Broadcasting a (3,) array against a (2, 3) one. NumPy compares the shapes from the right: 3 matches 3, and the absent first dimension is stretched to 2. The hollow row is not stored — it is the same three numbers, reused.

```
v.shape           (3,)
v[:, None].shape  (3, 1)
(v - v).shape     (3,)
(v[:, None] - v).shape (3, 3)  <-- 3x3, not 3
```



three numbers in, nine out — every pairwise difference, and no error

Figure 2: Why (3,) and (3, 1) are not interchangeable. Comparing from the right, a column of three against a row of three makes *both* dimensions stretch, and three numbers become nine. Nothing is raised.

The last line is the trap. You asked for differences between 3 numbers and got a 3×3 table of *all pairwise* differences, with no error raised. If a calculation downstream then averages it, you get a number — the wrong number. **Print the shape whenever you are unsure.**

Reductions and axis

A reduction collapses an array. With no axis it collapses everything to a single value:

```
table = np.arange(6).reshape(2, 3)
print(table)
print("sum of everything:", table.sum())
```

```
[[0 1 2]
```

```
[3 4 5]]
sum of everything: 15
```

With an axis it collapses only that direction. This is the single most misremembered thing in NumPy, so it is worth stating carefully:

`axis=0` means **collapse the row index**. You end up with one result per column.

```
print("table.sum(axis=0) ->", table.sum(axis=0), " shape",
      "\n  table.sum(axis=0).shape)
print("table.sum(axis=1) ->", table.sum(axis=1), " shape",
      "\n  table.sum(axis=1).shape)
```

```
table.sum(axis=0) -> [3 5 7]  shape (3,)
table.sum(axis=1) -> [ 3 12]  shape (2,)
```

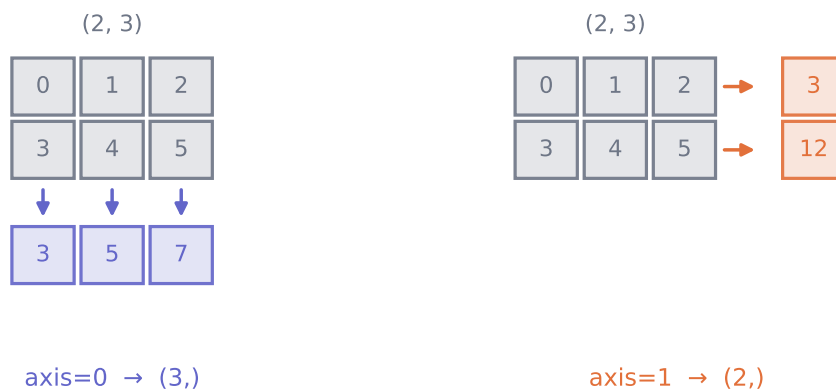


Figure 3: The axis you name is the one that disappears. Reducing over `axis=0` collapses the row index and leaves one number per column; over `axis=1` it collapses the column index and leaves one per row.

The reliable way to remember it is by shape rather than by words. The table is (2, 3). Reducing over `axis=0` removes the first dimension and leaves (3,); reducing over `axis=1` removes the second and leaves (2,). *The axis you name is the one that disappears.*

ddof, and a default that will catch you

`np.std` and `np.var` divide by n by default. The sample standard deviation that statistics uses divides by $n - 1$, which you request with `ddof=1`:

```
values = np.array([2.0, 4.0, 4.0, 4.0, 5.0, 5.0, 7.0, 9.0])
print("population (ddof=0):", values.std())
print("sample      (ddof=1):", values.std(ddof=1))
```

```
population (ddof=0): 2.0
sample      (ddof=1): 2.138089935299395
```

Neither is wrong; they answer different questions. But NumPy's default is not the one statistics usually wants, and the gap is largest for exactly the small samples where it matters most. Decide deliberately and say which you used.

Random numbers

Create one generator, near the top of the script, and draw from it repeatedly:

```
rng = np.random.default_rng(2026)
print(rng.normal(loc=0, scale=1, size=4))      # bell-shaped: centre loc, spread
↳ scale (S6)
print(rng.integers(low=1, high=7, size=4))     # whole numbers 1 to 6: high is
↳ excluded
```

```
[-0.79312248  0.24057128 -1.89632635  1.39577171]
[4 3 5 5]
```

The seed makes the sequence repeatable, which is what lets these notes print the same numbers on your machine as on mine. Do **not** call `np.random.default_rng(2026)` again inside a loop: each call restarts the sequence from the same place, so every iteration draws the identical “random” value. (Calling `default_rng()` with no seed inside a loop does give different values each time — but then nothing is reproducible, which is its own problem.) Create one generator outside the loop and draw from it repeatedly.

* is not @

For arrays, `*` multiplies element by element and `@` performs matrix multiplication. They are different questions with different answers:

```
A = np.array([[1.0, 2.0], [3.0, 4.0]])
print("A * A =\n", A * A)
print("A @ A =\n", A @ A)
```

```
A * A =
[[ 1.  4.]
 [ 9. 16.]]
A @ A =
[[ 7. 10.]
 [15. 22.]]
```

If **M9** is still ahead of you, take this on trust for now: `*` is arithmetic on matching positions, `@` is the row-times-column operation from linear algebra.

Worked example

Standardise every numeric column of the cohort table.

Standardising means subtracting each column’s mean and dividing by its standard deviation, so that every column ends up centred at 0 with a spread of 1. It uses reductions and broadcasting together, and it is the single most common preprocessing step in data science.

```
import pandas as pd

cohort = pd.read_csv("data/cohort.csv")
columns = ["diagnostic_score", "hours_practice", "final_score"]
raw = cohort[columns].to_numpy()

# Keep only rows that are complete and in range. The cohort file contains a
# deliberate missing value and one impossible score; P9 deals with them
# properly, but arithmetic on a NaN silently poisons everything it touches.
# np.isfinite is False for nan and also for infinity (inf), so it rejects both.
valid = np.isfinite(raw).all(axis=1) & (raw[:, 2] >= 0) & (raw[:, 2] <= 100)
data = raw[valid]

print("kept", valid.sum(), "of", len(raw), "rows; shape", data.shape)
```

kept 238 of 240 rows; shape (238, 3)

Now the two lines that matter:

```
means = data.mean(axis=0)          # one per column -> shape (3,)
sds = data.std(axis=0, ddof=1)     # one per column -> shape (3,)

standardised = (data - means) / sds

print("column means:", np.round(means, 2))
print("column sds  :", np.round(sds, 2))
print("shapes:", data.shape, "-", means.shape, "->", standardised.shape)
```

```
column means: [62.43 16.42 65.27]
column sds  : [17.49  8.14  8.38]
shapes: (238, 3) - (3,) -> (238, 3)
```

Read what happened. `data` is (238, 3) and `means` is (3,). Comparing from the right, 3 matches 3, and the missing first dimension is stretched to 238. One mean was subtracted from each column, exactly as intended.

Now check it, rather than assuming:

```
# np.abs gives the absolute value: the size, ignoring the sign (-3 becomes 3)
print("largest remaining mean:", np.abs(standardised.mean(axis=0)).max())
print("standard deviations  :", standardised.std(axis=0, ddof=1))
```

```
largest remaining mean: 2.1593371348696847e-15
standard deviations   : [1.  1.  1.]
```

The means are not exactly zero but around 10^{-15} , which is floating-point noise rather than a mistake — see [P2](#).

Two mistakes were available here, and only one of them would have announced itself. Using `axis=1` would have computed one mean per *student*, shape (238,), and subtracting that from a (238, 3)

array raises a `ValueError` — 238 against 3 cannot broadcast. That one fails loudly, which is the good case. Forgetting `ddof=1` is the quiet one. It divides by a slightly *smaller* number, so the standardised columns come out with standard deviations of 1.002 rather than 1 — the factor is $\sqrt{n/(n-1)}$. Nothing raises, the shapes are all correct, and the result looks entirely reasonable. Two lines of checking catch it.

Try it yourself

Exercise 1 `a` has shape `(4, 3)` and `b` has shape `(3,)`. What is the shape of `a * b`? What about `a * b[:, None]` — and why?

💡 Show the solution

`a * b` has shape `(4, 3)`. Comparing from the right: 3 against 3 matches, and `b`'s missing first dimension stretches to 4. Each row of `a` is multiplied by `b`.

`a * b[:, None]` **raises an error**. `b[:, None]` has shape `(3, 1)`. Comparing from the right: 1 against 3 is fine (the 1 stretches), but then 3 against 4 is not — neither is 1 and they are unequal. This one at least fails loudly. The dangerous version is when both dimensions happen to be compatible and you get a silently larger array.

Exercise 2 You have marks with shape `(30, 5)` — thirty students, five assignments. Write the expression for each student's average, and the expression for each assignment's average. State the shape of each result.

💡 Show the solution

Per student, average across the five assignments — collapse the *column* index:

```
marks.mean(axis=1)      # shape (30,)
```

Per assignment, average across the thirty students — collapse the *row* index:

```
marks.mean(axis=0)      # shape (5,)
```

Check them by shape rather than by intuition. If you wanted one number per student you should get 30 of them, and only `axis=1` does that.

Exercise 3 This is meant to centre each column, but it does not. Find the bug without running it, then say what shape the result has.

```
data = np.arange(12).reshape(4, 3)
centred = data - data.mean(axis=1)
```

💡 Show the solution

`data.mean(axis=1)` collapses the column index, giving one mean per **row** — shape `(4,)`. To centre columns you need one mean per column, `axis=0`, shape `(3,)`.

Here you are lucky: it raises. Comparing `(4, 3)` with `(4,)` from the right, 4 against 3 is neither equal nor 1, so NumPy refuses and you get a `ValueError`. There is no result shape.

Now change the array to `np.arange(16).reshape(4, 4)` and the luck runs out. `(4, 4)` against

(4,) broadcasts happily along the last axis, so the value at position `[i, j]` has the mean of **row j** subtracted from it — the row means get lined up against *columns*. That is not row-centring and it is not column-centring; it is nothing at all. You get a plausible (4, 4) array of meaningless numbers.

The fix for centring columns is `data - data.mean(axis=0)`. If you genuinely wanted to centre rows, you need `data - data.mean(axis=1, keepdims=True)`, where `keepdims=True` gives shape (4, 1) so the means line up against rows.

The lesson: square arrays hide axis errors, so be most careful exactly when the shape looks tidiest.

Check it in Python

Two habits are worth building deliberately. The first is asserting the shape you expect, so a broadcast that goes wrong stops the program instead of quietly continuing:

```
values = np.array([10.0, 20.0, 30.0])

differences = values - values.mean()
assert differences.shape == values.shape, f"expected {values.shape}, got
    {differences.shape}"
print("shape as expected:", differences.shape)
```

```
shape as expected: (3,)
```

The second is checking that a transformation did what you claimed. Here is the axis bug from Exercise 3, in the square case where it does not raise:

```
data = np.arange(16, dtype=float).reshape(4, 4)

wrong = data - data.mean(axis=1)      # one mean per row - wrong direction
right = data - data.mean(axis=0)      # one mean per column - what we wanted

print("both shapes:", wrong.shape, right.shape)
print("column means after 'wrong':", np.round(wrong.mean(axis=0), 3))
print("column means after 'right':", np.round(right.mean(axis=0), 3))
```

```
both shapes: (4, 4) (4, 4)
column means after 'wrong': [ 4.5  1.5 -1.5 -4.5]
column means after 'right': [0.  0.  0.  0.]
```

Both produced a (4, 4) array and neither complained. In `wrong`, the row means were broadcast against columns — the mean of row 1 subtracted from column 1, and so on — which is not a meaningful operation at all. Only the check distinguishes them: after centring columns correctly every column mean must be zero, and in the wrong version they plainly are not.

If you take one thing from this lesson, make it that. **Predict the shape, then verify the property.** It costs one line and it catches the entire class of error that NumPy will not catch for you.

Common wrong turns

Where this usually goes wrong

Reading `axis=0` as “along the rows” The phrasing traps everyone. `axis=0` collapses the row index and gives one result per column. Think in shapes: the axis you name is the one that disappears.

Confusing `(n,)` with `(n, 1)` A flat array of length n and a column of n values behave completely differently under broadcasting. Combining the two shapes gives an $n \times n$ result, usually without an error.

Trusting a result because it has a plausible shape NumPy checks that shapes are *compatible*, never that they are *meaningful*. A square array will absorb an axis mistake in total silence.

Leaving `ddof` at its default `np.std` divides by n . The sample standard deviation divides by $n - 1$. Choose on purpose and record which you used.

Reseeding inside a loop `np.random.default_rng(42)` created afresh on each iteration restarts the same sequence, so every “random” draw is identical. Create the generator once, outside the loop, and pass it in.

Using `*` when you meant `@` For arrays, `*` multiplies matching positions and `@` does matrix multiplication. Both may succeed on the same inputs and return different answers.

Letting a NaN into the arithmetic A single missing value propagates through every sum, mean and product it reaches, turning the whole result into `nan`. Filter first, and be able to say how many rows you dropped and why.

Check yourself

Answers are at the foot of the page.

1. `x` has shape `(5, 8)`. What is the shape of `x.sum(axis=1)`?
 - a. `(8,)`
 - b. `(5,)`
 - c. `(5, 1)`
 - d. a single number
2. `a` has shape `(3, 4)` and `b` has shape `(4,)`. Does `a + b` work? What about `a + b[:, None]`?
3. Why can an axis mistake on a square array be more dangerous than the same mistake on a rectangular one?
4. A colleague reports a standard deviation of 8.37 where you calculated 8.38 from the same numbers. What is the most likely explanation?
5. A loop draws one random number per iteration and every value comes out the same. What is the most likely cause?



Show the answers

1. **(b) `(5,)`**. `axis=1` removes the second dimension, leaving one result per row. Option (a) is what `axis=0` gives.
2. `a + b` works: comparing from the right, 4 matches 4, and the missing dimension stretches to 3. `a + b[:, None]` has shape `(4, 1)` against `(3, 4)`; from the right, 1 stretches to 4, but then 4 against 3 fails, so it raises a `ValueError`.

3. Because the shapes stay compatible either way. On a (4, 3) array, using the wrong axis usually raises an error; on a (4, 4) array both directions broadcast happily and you get a correctly shaped, wrong answer with no warning.
4. One of you used `ddof=1` and the other did not — 8.37 is the n divisor and 8.38 the $n - 1$ divisor. Neither is a mistake, but the two answer different questions and the report should say which.
5. The generator is being created inside the loop **with a fixed seed**, so the sequence restarts from the same place on every iteration. Move `rng = np.random.default_rng(2026)` outside the loop. (An unseeded `default_rng()` inside the loop would not repeat — but it would also not be reproducible, so the fix is the same.)

Where this shows up next

P8 moves to labelled tables, where columns have names and types instead of positions — but the arithmetic underneath is everything in this lesson. **P10** plots the arrays you build here. **M9** explains what `@` is actually computing, and why the shape rules are what they are.

pandas: the DataFrame

Unit 3 · Lesson 1 of 4 · Python · P8 Reading: 6 pages With practice: 2 hours Assumes: P7

Why this lesson is here

Real data is a **table**: columns with names, columns with different types, values that are missing. A NumPy array cannot express that — it insists every element share one type, and it identifies columns by number, so P6 had you writing `raw[:, 2]` and remembering what 2 meant.

pandas keeps NumPy underneath and adds the two things a table needs: **names** and **mixed types**. It is the tool you will touch every single day, which is why it gets two lessons.

Before you start

You should be able to:

- work with arrays, shapes and boolean masks — P6, P7;
- say what a dtype is and why it matters — P6;
- write a function and an assertion — P5.

Quick check. In `raw[:, 2]`, what does the 2 refer to, and what breaks if a column is inserted before it?

💡 Show the answer

The third column, by position. Insert a column anywhere before it and 2 now means something else — silently. That fragility is most of the reason this lesson exists.

The idea

```
import pandas as pd

cohort = pd.read_csv("data/cohort.csv")
print(cohort.shape)
cohort.head()
```

(240, 7)

	student_id	background	diagnostic_score	hours_practice	final_score	completed	started
0	S0001	Mathematics or Physics	89.1	12.9	83.6	True	2026-0
1	S0002	Life sciences	42.4	22.8	56.6	True	2026-0
2	S0003	Engineering	68.4	1.7	58.5	True	2026-0
3	S0004	Computing	64.3	7.5	67.5	True	2026-0
4	S0005	Computing	69.0	13.5	73.8	True	2026-0

A **DataFrame** is a table. Each column is a **Series** — an array with a name and a dtype of its own — and all the columns share an **index** identifying the rows.

	student_id	background	final_score
	index object	object	float64
0	S0001	Maths...	83.6
1	S0002	Life sci...	56.6
2	S0003	Eng...	58.5

one dtype per column — not one for the whole table

Figure 1: The three parts of a DataFrame: an index naming the rows, column labels naming the columns, and a dtype per column. The dtype is per column, which is what a NumPy array cannot do.

The mechanics

The four things you always do after loading

```
print(cohort.shape)
print()
print(cohort.dtypes)
print()
print(cohort.isna().sum())
```

(240, 7)

```
student_id      object
background      object
diagnostic_score float64
hours_practice  float64
final_score      float64
completed        bool
started_on      object
dtype: object
```

```
student_id      0
background      1
diagnostic_score 0
hours_practice  0
final_score      1
completed        0
started_on      0
dtype: int64
```

Shape tells you whether you loaded what you expected. dtypes tells you what pandas *decided* each column was. `isna().sum()` counts missing values per column. Doing these three before anything else takes ten seconds and catches most loading disasters.

Notice `final_score` came in as `float64` and `background` as `object` — pandas' name for "text, or mixed". Notice also that one `final_score` and one `background` are missing, which matches the file's planted defects.

Never trust the inferred type

pandas guesses well and is not always right. Convert deliberately, and let it fail loudly if the data disagrees:

```
cohort["final_score"] = pd.to_numeric(cohort["final_score"], errors="raise")
cohort["started_on"] = pd.to_datetime(cohort["started_on"], errors="raise")
print(cohort[["final_score", "started_on"]].dtypes)
```

```
final_score          float64
started_on          datetime64[ns]
dtype: object
```

`errors="raise"` is the important part. The alternative, `errors="coerce"`, silently turns anything unparseable into `NaT/NaN` — convenient, and a good way to lose data without noticing. Use `raise` first; reach for `coerce` only when you have decided that unparseable values *should* become missing, and then count how many did.

Selecting columns and rows

```
print(type(cohort["final_score"]).__name__)      # one column -> Series
print(type(cohort[["final_score", "background"]]).__name__)  # a list ->
↳ DataFrame
```

```
Series
DataFrame
```

One name gives a `Series`; a *list* of names gives a `DataFrame`. The doubled brackets are not a typo — they are a list inside the selection.

For rows, use `.loc` (by label) or `.iloc` (by position):

```
print(cohort.loc[0, "student_id"])      # label 0, that column
print(cohort.iloc[0, 0])                # first row, first column
cohort.loc[0:2, ["student_id", "final_score"]]
```

```
S0001
S0001
```

	student_id	final_score
0	S0001	83.6
1	S0002	56.6
2	S0003	58.5

! `.loc` slices are **inclusive** of the end label — `loc[0:2]` gives three rows. Everywhere else in Python the end is excluded. It is an inconsistency worth knowing rather than being caught by; `.iloc[0:2]` gives two rows as you would expect.

Selecting rows by condition

```
high = cohort[cohort["final_score"] >= 75]
print(f"{len(high)} students scored 75 or more")

quantitative = cohort[cohort["background"].isin(["Computing", "Engineering"])]
print(f"{len(quantitative)} from computing or engineering")
```

25 students scored 75 or more
106 from computing or engineering

Combine conditions with `&` and `|`, each in its own parentheses — the same rule as P6:

```
selected = cohort[(cohort["final_score"] >= 60) & (cohort["hours_practice"] <
    - 15)]
print(f"{len(selected)} scored 60+ while practising under 15 hours")
```

98 scored 60+ while practising under 15 hours

Adding a column

```
cohort["score_per_hour"] = cohort["final_score"] / cohort["hours_practice"]
print(cohort[["final_score", "hours_practice", "score_per_hour"]].head(3))
```

	final_score	hours_practice	score_per_hour
0	83.6	12.9	6.480620
1	56.6	22.8	2.482456
2	58.5	1.7	34.411765

The arithmetic is elementwise across the whole column, exactly as in P7 — pandas is NumPy underneath.

The trap that catches everyone

```
subset = cohort[cohort["final_score"] >= 75]
subset["flag"] = True          # pandas may or may not be modifying a copy
print(f"{len(subset)} rows flagged")
```

25 rows flagged

Selecting rows may give you a view or a copy, and pandas does not promise which. Writing to the result is therefore unreliable — sometimes it silently fails to stick, sometimes it warns. The fix is to say what you want:

```
subset = cohort[cohort["final_score"] >= 75].copy()
subset["flag"] = True
print("independent copy:", len(subset), "rows,", subset["flag"].all())
```

independent copy: 25 rows, True

Rule: if you select rows and then write to the result, add `.copy()`. It is one word and it removes an entire category of confusion.

Worked example

Loading the cohort into a validated analysis table.

```
from pathlib import Path

def load_cohort(path):
    """Load the cohort file, convert types deliberately, and validate the
    ↪ schema."""
    frame = pd.read_csv(path)

    # the schema: the exact set of column names the file must have
    expected = {"student_id", "background", "diagnostic_score",
                "hours_practice", "final_score", "completed", "started_on"}
    if set(frame.columns) != expected:
        # a ^ b gives the names in one set but not the other
        raise ValueError(f"unexpected columns: {sorted(set(frame.columns) ^
    ↪ expected)}")

    for column in ("diagnostic_score", "hours_practice", "final_score"):
        frame[column] = pd.to_numeric(frame[column], errors="raise")
    frame["started_on"] = pd.to_datetime(frame["started_on"], errors="raise")

    return frame

cohort = load_cohort(Path("data/cohort.csv"))
print(cohort.shape)
print(cohort.dtypes)
```

```
(240, 7)
student_id          object
background          object
diagnostic_score    float64
hours_practice      float64
final_score         float64
```

```

completed                                bool
started_on                             datetime64[ns]
dtype: object

```

Now split it into what you will analyse and what you will not — keeping both:

```

valid = (
    cohort["final_score"].between(0, 100)
    & cohort["final_score"].notna()
    & cohort["background"].notna()
)

analysis = cohort.loc[valid].copy()
excluded = cohort.loc[~valid].copy()

print(f"analysis: {len(analysis)} rows")
print(f"excluded: {len(excluded)} rows")
excluded[["student_id", "background", "final_score"]]

```

```

analysis: 237 rows
excluded: 3 rows

```

	student_id	background	final_score
17	S0018	Computing	NaN
102	S0103	NaN	66.2
188	S0189	Life sciences	187.0

Three rows excluded, and you can see exactly which and why: one missing score, one missing background, one impossible score of 187. That table is a result, not a byproduct — P9 writes it to a file.

A last check that the split lost nothing:

```

assert len(analysis) + len(excluded) == len(cohort)
assert analysis["final_score"].between(0, 100).all()
assert analysis["background"].notna().all()
print(f"{len(analysis)} + {len(excluded)} = {len(cohort)}; all conditions
↳ verified")

```

```
237 + 3 = 240; all conditions verified
```

Try it yourself

Exercise 1 What is the difference between `cohort["final_score"]` and `cohort[["final_score"]]`?

 Show the solution

The first gives a **Series** — one column, one dimension. The second gives a **DataFrame** with one column, because the inner brackets are a *list* of column names.

It matters when the next step expects one or the other. `.mean()` on a Series gives a number; on a DataFrame it gives a Series with one entry.

Exercise 2 This is meant to flag high scorers. It does not reliably work, and it never changes cohort. Why? Fix it twice: once for a separate table of high scorers, once for a flag in cohort itself.

```
high = cohort[cohort["final_score"] >= 80]
high["distinction"] = True
```

 Show the solution

`high` is a new object selected out of `cohort`, so writing to it does not touch `cohort` — and because pandas does not promise whether it is a view or a copy, even the write to `high` is unreliable.

If you wanted an independent table of high scorers:

```
high = cohort[cohort["final_score"] >= 80].copy()
high["distinction"] = True
```

If you wanted to flag rows **in the original**, assign through `.loc` in one statement:

```
cohort["distinction"] = False
cohort.loc[cohort["final_score"] >= 80, "distinction"] = True
```

The second is usually what people actually meant.

Exercise 3 Write one expression giving the mean `diagnostic_score` of students from "Life sciences" whose `final_score` is recorded. Then say why the second condition is needed.

 Show the solution

```
mask = (cohort["background"] == "Life sciences") &
    ~ cohort["final_score"].notna()
cohort.loc[mask, "diagnostic_score"].mean()
```

If you said “it keeps only students whose final score is recorded”, that is what it does; a full answer also says why that matters here, as follows.

Strictly the `notna()` is not needed to compute a mean *of diagnostic scores* — pandas skips missing values automatically, and the missing value is in a different column anyway.

It is needed because of what the number will be compared with. If you later report this alongside a mean final score computed on a different set of rows, the two summarise different students. Fixing the analysis set once, up front, is what the worked example does and why.

Check it in Python

Check the schema, not just the shape. A file with the right number of columns in the wrong order will load without complaint:

```
expected = ["student_id", "background", "diagnostic_score",
            "hours_practice", "final_score", "completed", "started_on"]
assert list(cohort.columns) == expected, f"got {list(cohort.columns)}" # fails
↳ if you added `distinction` in Exercise 2 – reload first
print("schema as expected:", len(expected), "columns in order")
```

schema as expected: 7 columns in order

Check that a filter conserves rows, so nothing vanishes unaccounted for:

```
assert len(analysis) + len(excluded) == len(cohort)
print(f"{len(analysis)} kept + {len(excluded)} excluded = {len(cohort)} total")
```

237 kept + 3 excluded = 240 total

Check the ranges you assumed, rather than hoping:

```
summary = analysis[["diagnostic_score", "hours_practice", "final_score"]].agg(
    ["min", "max", "mean"]
).round(2)
print(summary)

assert analysis["final_score"].between(0, 100).all()
assert (analysis["hours_practice"] > 0).all()
print("\nall ranges plausible")
```

	diagnostic_score	hours_practice	final_score
min	27.20	1.00	44.50
max	100.00	36.20	91.60
mean	62.51	16.38	65.27

all ranges plausible

If hours_practice ever came back negative, that assertion would fire before the number reached a plot or a model.

Common wrong turns

Where this usually goes wrong

Writing to the result of a row selection `frame[mask]["col"] = x` may modify a temporary copy and vanish. Use `.copy()` for an independent table, or `frame.loc[mask, "col"] = x` to write into the original.

Trusting the inferred dtype A column of numbers with one stray "missing" loads as object, and every arithmetic operation on it then behaves strangely. Look at `.dtypes` after every load.

Using `errors="coerce"` as a default It converts anything unparseable to missing, silently. Use `raise` first, and only `coerce` once you have decided that is the right treatment — then count how many were affected.

Forgetting `.loc` slices include the end label `loc[0:2]` gives three rows; `iloc[0:2]` gives two. The inconsistency is real; knowing it is easier than being surprised by it.

Selecting on one column and summarising another If the row set differs between two numbers you intend to compare, they are about different students. Fix the analysis set once, up front.

Dropping rows without keeping them An excluded row is a decision that should be inspectable. Keep the excluded table; do not just filter it away.

Check yourself

Answers are at the foot of the page.

1. `cohort[["final_score"]]` returns what?
 - a. a Series
 - b. a DataFrame
 - c. a NumPy array
 - d. a list
2. How many rows does `frame.loc[0:2]` return, and how many does `frame.iloc[0:2]`?
3. Why does `pd.to_numeric(col, errors="coerce")` deserve caution?
4. A numeric column loads with dtype object. What is the likely cause?
5. You filter to high scorers and add a flag, but the original table is unchanged and no error appeared. What happened?

Show the answers

1. **(b) a DataFrame** with one column. The inner brackets make it a list of names, and a list always gives a DataFrame.
2. `.loc[0:2]` gives **three** (labels 0, 1 and 2 — the end is included); `.iloc[0:2]` gives **two** (positions 0 and 1, the usual exclusive end).
3. It converts every unparseable value to NaN without saying so, so a column that is 30% unparseable looks like a column that is 30% missing. Use `raise` until you have decided otherwise, then count the coerced values.
4. At least one value is not a number — a word such as "missing", a thousands separator, or a stray unit like "22C". (Common markers such as "n/a" and empty cells are read as missing automatically, and do not do this.) Find it before converting.
5. The selection produced a separate object, so the flag was written there and not into the original. Use `frame.loc[mask, "flag"] = True` to write into the original, or `.copy()` if you wanted a separate table.

Where this shows up next

P9 takes the excluded table seriously, groups the analysis table, and joins it to a second file. **P10** plots

these columns. **S2** and **S3** use exactly this table to describe one and two variables.

pandas: cleaning, grouping, joining

Unit 3 · Lesson 2 of 4 · Python · P9 Reading: 8 pages With practice: 2 hours Assumes: P8

Why this lesson is here

Three operations account for most of the work in most analyses: deciding what to do about missing values, summarising by group, and combining two tables.

None of them is technically hard. All three are easy to get **quietly** wrong — a `dropna()` that removes a fifth of your data, a join that duplicates rows, a group mean computed over a different set of rows than you thought. This lesson is about making each of those decisions visible.

Before you start

You should be able to:

- load a table, check its dtypes, and select rows by condition — P8;
- explain why `.copy()` matters after a row selection — P8;
- write an assertion — P5.

Quick check. After `subset = frame[frame["x"] > 0]`, why might writing to `subset` not affect `frame`?

💡 Show the answer

Because the selection produced a separate object. pandas does not promise whether it is a view or a copy, which is why you add `.copy()` when you intend an independent table and use `frame.loc[mask, col] = ...` when you intend to write into the original.

The idea

```
import pandas as pd
from pathlib import Path

cohort = pd.read_csv("data/cohort.csv")
for column in ("diagnostic_score", "hours_practice", "final_score"):
    cohort[column] = pd.to_numeric(cohort[column], errors="raise")

print(cohort.shape)
print(cohort.isna().sum())
```

```
(240, 7)
student_id      0
background      1
diagnostic_score 0
hours_practice  0
final_score     1
```

```
completed          0
started_on         0
dtype: int64
```

Two missing values, in different columns and different rows. Before touching them, ask the question this lesson turns on: **why is it missing?**

- The score was never recorded because the student withdrew — the missingness is related to the outcome, and dropping the row biases the result.
- The score was lost in a transcription error — unrelated to the outcome, and dropping is comparatively harmless.
- The field does not apply to that student at all — structurally missing, and the row may belong to a different population.

pandas cannot tell you which. `dropna()` will happily proceed under all three, which is why it is a **decision**, not a cleaning step.

The mechanics

Look at missingness before removing it

```
missing_by_group = (
    # .assign adds a True/False column, no_score; groupby is explained below
    cohort.assign(no_score=cohort["final_score"].isna())
    .groupby("background", dropna=False)["no_score"]
    .agg(["sum", "size", "mean"])
    .rename(columns={"sum": "missing", "size": "n", "mean": "proportion"})
)
print(missing_by_group)
```

	missing	n	proportion
background			
Computing	1	70	0.014286
Engineering	0	36	0.000000
Life sciences	0	58	0.000000
Mathematics or Physics	0	36	0.000000
Social sciences	0	39	0.000000
NaN	0	1	0.000000

One missing value, in one group. If instead it had been forty, all in the lowest scoring group, **complete-case analysis** — using only the rows with nothing missing in the columns you analyse — would systematically overstate the cohort mean — and the number would look perfectly reasonable.

Note `dropna=False` in the `groupby`: without it, the row whose *background* is missing would vanish from this very summary, which is the opposite of what a missingness report is for.

Split, and keep both halves

```

valid = (
    cohort["final_score"].between(0, 100)
    & cohort["final_score"].notna()
    & cohort["background"].notna()
)

analysis = cohort.loc[valid].copy()
excluded = cohort.loc[~valid].copy()
# itertuples() gives one row at a time; r.final_score is that row's value
excluded["reason"] = [
    "final_score missing" if pd.isna(r.final_score)
    else "background missing" if pd.isna(r.background)
    else f"final_score out of range ({r.final_score})"
    for r in excluded.itertuples()
]

print(f"{len(analysis)} for analysis, {len(excluded)} excluded")
excluded[["student_id", "background", "final_score", "reason"]]

```

237 for analysis, 3 excluded

	student_id	background	final_score	reason
17	S0018	Computing	NaN	final_score missing
102	S0103	NaN	66.2	background missing
188	S0189	Life sciences	187.0	final_score out of range (187.0)

The exclusions table with reasons is the deliverable. “We dropped three rows” is not reproducible; this is.

Grouping: split, apply, combine

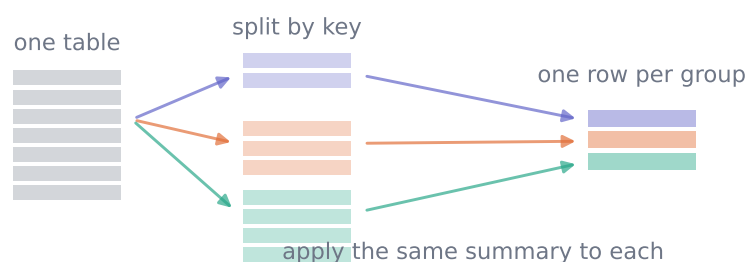


Figure 1: Split–apply–combine. The rows are split by a key, the same summary is applied to each part, and the results are stacked back into one table with one row per group.

```

summary = (
    analysis.groupby("background", as_index=False)

```

```

    .agg(n=("final_score", "size"),
         mean_score=("final_score", "mean"),
         sd_score=("final_score", "std"),
         mean_hours=("hours_practice", "mean"))
    .round(2)
)
summary

```

	background	n	mean_score	sd_score	mean_hours
0	Computing	69	65.85	6.60	12.47
1	Engineering	36	67.15	5.33	10.45
2	Life sciences	57	60.51	7.29	22.61
3	Mathematics or Physics	36	75.28	7.63	8.03
4	Social sciences	39	60.21	6.58	27.37

`as_index=False` keeps background as an ordinary column rather than moving it into the index, which makes what follows simpler. The named arguments to `.agg` give each output column an explicit name, so you never have to decode a multi-level heading.

pandas' `.std()` uses the $n - 1$ divisor by default — the sample standard deviation, the opposite of NumPy's default. That inconsistency is real and worth remembering; [P7](#) covers the NumPy side.

Joining, with the relationship checked

```

meta = pd.read_csv("data/background_meta.csv")
print(meta)

```

	background	quantitative	typical_prior_maths
0	Mathematics or Physics	True	A-level Further Maths or degree-level
1	Computing	True	A-level Maths
2	Engineering	True	A-level Maths
3	Life sciences	False	GCSE, some statistics
4	Social sciences	False	GCSE

```

combined = summary.merge(
    meta,
    on="background",
    how="left",
    validate="one_to_one",
    indicator=True,
)
print(combined["_merge"].value_counts().to_string())
combined = combined.drop(columns="_merge")
combined

```

_merge

both

5

```
left_only    0
right_only   0
```

	background	n	mean_score	sd_score	mean_hours	quantitative	typical_prior_maths
0	Computing	69	65.85	6.60	12.47	True	A-level Maths
1	Engineering	36	67.15	5.33	10.45	True	A-level Maths
2	Life sciences	57	60.51	7.29	22.61	False	GCSE, some statistics
3	Mathematics or Physics	36	75.28	7.63	8.03	True	A-level Further Maths or
4	Social sciences	39	60.21	6.58	27.37	False	GCSE

`how="left"` keeps every row of the left table, summary, whether it matches or not; rows of meta with no match are dropped. Two more arguments do the work here, and they should be habits.

`validate="one_to_one"` states the relationship you believe holds — one summary row per background, one metadata row per background — and pandas **raises** if it does not. A join that unexpectedly matches many-to-many silently multiplies your rows and changes every downstream number.

`indicator=True` adds a column saying, for each row, whether it matched on both sides. Checking it catches the other failure: a key that quietly matched nothing and left NaN where you expected data.

```
broken = pd.concat([meta, meta.head(1)])          # a duplicated key
summary.merge(broken, on="background", how="left", validate="one_to_one")
```

MergeError: Merge keys are not unique in right dataset; not a one-to-one merge

```
-----
MergeError                                Traceback (most recent call last)
Cell In[8], line 2
      1 broken = pd.concat([meta, meta.head(1)])          # a duplicated key
----> 2 summary.merge(broken, on="background", how="left", validate="one_to_one")
File ~/anaconda3/lib/python3.11/site-packages/pandas/core/frame.py:10859, in
DataFrame.merge(self, right, how, on, left_on, right_on, left_index,
right_index, sort, suffixes, copy, indicator, validate)
    10840 @Substitution("")
    10841 @Appender(_merge_doc, indents=2)
    10842 def merge(
    (...) 10855     validate: MergeValidate | None = None,
    10856 ) -> DataFrame:
    10857     from pandas.core.reshape.merge import merge
> 10859     return merge(
    10860         self,
    10861         right,
    10862         how=how,
    10863         on=on,
    10864         left_on=left_on,
    10865         right_on=right_on,
    10866         left_index=left_index,
    10867         right_index=right_index,
    10868         sort=sort,
    10869         suffixes=suffixes,
    10870         copy=copy,
    10871         indicator=indicator,
```

```

10872         validate=validate,
10873     )
File ~/anaconda3/lib/python3.11/site-packages/pandas/core/reshape/merge.py:170,
  in merge(left, right, how, on, left_on, right_on, left_index, right_index,
  sort, suffixes, copy, indicator, validate)
    155     return _cross_merge(
    156         left_df,
    157         right_df,
    (...) 167         copy=copy,
    168     )
    169 else:
--> 170     op = _MergeOperation(
    171         left_df,
    172         right_df,
    173         how=how,
    174         on=on,
    175         left_on=left_on,
    176         right_on=right_on,
    177         left_index=left_index,
    178         right_index=right_index,
    179         sort=sort,
    180         suffixes=suffixes,
    181         indicator=indicator,
    182         validate=validate,
    183     )
    184     return op.get_result(copy=copy)
File ~/anaconda3/lib/python3.11/site-packages/pandas/core/reshape/merge.py:813,
  in _MergeOperation.__init__(self, left, right, how, on, left_on, right_on,
  left_index, right_index, sort, suffixes, indicator, validate)
    809 # If argument passed to validate,
    810 # check if columns specified as unique
    811 # are in fact unique.
    812 if validate is not None:
--> 813     self._validate_validate_kwd(validate)
File ~/anaconda3/lib/python3.11/site-packages/pandas/core/reshape/merge.py:1658,
  in _MergeOperation._validate_validate_kwd(self, validate)
    1654     raise MergeError(
    1655         "Merge keys are not unique in left dataset; not a one-to-one
  merge"
    1656     )
    1657     if not right_unique:
-> 1658     raise MergeError(
    1659         "Merge keys are not unique in right dataset; not a one-to-one
  merge"
    1660     )
    1662 elif validate in ["one_to_many", "1:m"]:
    1663     if not left_unique:
MergeError: Merge keys are not unique in right dataset; not a one-to-one merge

```

That is the check earning its place.

Long and wide

```
wide = summary[["background", "mean_score", "mean_hours"]]
long = wide.melt(id_vars="background", var_name="measure", value_name="value")
print(long.head(6))
```

	background	measure	value
0	Computing	mean_score	65.85
1	Engineering	mean_score	67.15
2	Life sciences	mean_score	60.51
3	Mathematics or Physics	mean_score	75.28
4	Social sciences	mean_score	60.21
5	Computing	mean_hours	12.47

Wide is easier to read; long is easier to plot and group. `melt` goes one way, `pivot` the other. Most plotting libraries, and most statistical models, want long.

Worked example

The whole pipeline, with every decision recorded.

```
OUT = Path("_outputs")
OUT.mkdir(exist_ok=True)

# 1. load and convert deliberately
raw = pd.read_csv("data/cohort.csv")
for column in ("diagnostic_score", "hours_practice", "final_score"):
    raw[column] = pd.to_numeric(raw[column], errors="raise")

# 2. split into analysis and exclusions, keeping both
valid = (raw["final_score"].between(0, 100)
        & raw["final_score"].notna()
        & raw["background"].notna())
analysis, excluded = raw.loc[valid].copy(), raw.loc[~valid].copy()
excluded["reason"] = [
    # the same reasons as in 9.3
    "final_score missing" if pd.isna(r.final_score)
    else "background missing" if pd.isna(r.background)
    else f"final_score out of range ({r.final_score})"
    for r in excluded.itertuples()
]

# 3. summarise by group
summary = (analysis.groupby("background", as_index=False)
            .agg(n=("final_score", "size"),
                 mean_score=("final_score", "mean"),
                 sd_score=("final_score", "std"))
            .round(2))

# 4. join metadata, with the relationship asserted
```

```

meta = pd.read_csv("data/background_meta.csv")
report = summary.merge(meta, on="background", how="left",
                       validate="one_to_one", indicator=True)
assert (report["_merge"] == "both").all(), "a group had no metadata"
report = report.drop(columns="_merge")

# 5. write both results
report.to_csv(OUT / "group_summary.csv", index=False)
excluded.to_csv(OUT / "excluded_rows.csv", index=False)

print(f"{len(analysis)} analysed, {len(excluded)} excluded")
print(f"wrote {OUT/'group_summary.csv'} and {OUT/'excluded_rows.csv'}")
report

```

237 analysed, 3 excluded

wrote _outputs/group_summary.csv and _outputs/excluded_rows.csv

	background	n	mean_score	sd_score	quantitative	typical_prior_maths
0	Computing	69	65.85	6.60	True	A-level Maths
1	Engineering	36	67.15	5.33	True	A-level Maths
2	Life sciences	57	60.51	7.29	False	GCSE, some statistics
3	Mathematics or Physics	36	75.28	7.63	True	A-level Further Maths or degree-level
4	Social sciences	39	60.21	6.58	False	GCSE

Five steps, and the two that people skip are steps 2 and 4. Keeping the excluded rows means someone can ask *which ones?* and get an answer. Validating the join means the row count cannot silently change under you.

```

assert len(analysis) + len(excluded) == len(row)
assert report["n"].sum() == len(analysis)
print(f"{report['n'].sum()} students across {len(report)} groups "
      f"={len(analysis)} analysed. Nothing lost, nothing duplicated.")

```

237 students across 5 groups = 237 analysed. Nothing lost, nothing duplicated.

That last assertion is the one that catches a bad join: if the merge had duplicated rows, the group counts would no longer add up to the number of students.

Try it yourself

Exercise 1 `frame.dropna()` removes 40% of your rows. Name two things you would check before accepting that, and one before rejecting it.

 Show the solution

Before accepting:

- **Which columns** the missingness is in. `dropna()` with no arguments drops a row if *any* column is missing, including ones you never use. `subset=` limits it to the columns that matter.
- **Whether missingness relates to the outcome.** Group the missing indicator by anything you know about the rows, as in the `missing_by_group` table above. If missing rows differ systematically, complete-case analysis is biased.

Before rejecting: whether the rows are missing *structurally* — a field that does not apply to that kind of record. Those rows may simply not belong to the population you are describing, and removing them is correct.

Exercise 2 After a merge, your table has 340 rows where both inputs had 240. What happened, and which one argument would have caught it?

 Show the solution

The key is not unique on the right-hand side, so some left rows matched several right rows and were duplicated.

`validate="many_to_one"` — or `"one_to_one"` if you believe both sides are unique — makes pandas raise instead of silently multiplying.

Why it matters beyond tidiness: every duplicated row now contributes more than once to every mean, count and model that follows, and nothing about the output looks wrong.

Exercise 3 Produce a table of the mean `diagnostic_score` and mean `final_score` per background, sorted by mean final score descending. Then say what the ordering shows.

 Show the solution

```
(analysis.groupby("background", as_index=False)
 .agg(diagnostic=("diagnostic_score", "mean"),
      final=("final_score", "mean"))
 .round(2)
 .sort_values("final", ascending=False))
```

The ordering by final score is almost exactly the ordering by diagnostic score — groups that arrived ahead finished ahead. That is the confounding structure planted in this dataset, and it is why the overall relationship between practice and score points the wrong way. **S12** takes it apart properly.

Check it in Python

Assert that a split conserves rows:

```
assert len(analysis) + len(excluded) == len(row)
print(f"{len(analysis)} + {len(excluded)} = {len(row)}")
```

237 + 3 = 240

Assert that a join did not change the row count:

```
before = len(summary)
after = len(report)
assert before == after, f"merge changed row count: {before} -> {after}"
print(f"merge preserved {after} rows")
```

merge preserved 5 rows

Assert that group sizes add up to the analysis set — the check that actually catches a silent duplication:

```
assert report["n"].sum() == len(analysis)
print(f"group sizes sum to {report['n'].sum()}, matching {len(analysis)} analysed
↳ rows")
```

group sizes sum to 237, matching 237 analysed rows

And record what you excluded, in the output, not just in your head:

```
print(pd.read_csv(OUT / "excluded_rows.csv")[["student_id",
↳ "final_score"]].to_string(index=False))
```

student_id	final_score
S0018	NaN
S0103	66.2
S0189	187.0

Common wrong turns

Where this usually goes wrong

dropna() as a cleaning step It is a statistical decision about the population you are describing. Look at where the missingness is and whether it relates to the outcome first, and use `subset=` so unrelated columns cannot remove rows.

Merging without validate An unexpected many-to-many join multiplies rows and changes every number that follows, with no error. State the relationship and let pandas enforce it.

Not checking indicator A key that matched nothing leaves NaN rather than an error. The indicator column turns that into something you can assert on.

Assuming .std() matches NumPy pandas uses the $n - 1$ divisor by default, NumPy uses n . Two “standard deviations” of the same column can differ for this reason alone.

Grouping with dropna=True in a missingness report The rows whose key is missing disappear from the very summary meant to find them.

Discarding the excluded rows Then nobody, including you, can answer “which ones, and why?”. Write them out.

Check yourself

Answers are at the foot of the page.

1. `frame.dropna()` with no arguments removes a row when:
 - a. all columns are missing
 - b. any column is missing
 - c. the first column is missing
 - d. the index is missing
2. What does `validate="one_to_one"` protect against?
3. Why does `groupby(..., dropna=False)` matter when reporting missingness?
4. Group counts sum to 260 but your analysis table has 238 rows. What is the most likely explanation?
5. Give one situation where dropping missing rows is the right call, and one where it is not.

Show the answers

1. **(b) any column.** Which is why a column you never use can silently remove rows. Use `subset=` to name the columns that matter.
2. A join whose key is not unique where you assumed it was, which duplicates rows and changes every downstream number without raising.
3. Because rows whose grouping key is itself missing would be dropped from the report — hiding exactly the cases the report exists to find.
4. The merge duplicated rows: a key matched more than once on the right-hand side. `validate=` would have raised instead.
5. **Right:** the values are missing structurally — the field does not apply — so those rows are not part of the population being described. **Not right:** the values are missing *because of* the outcome, e.g. students who did badly withdrew before the final score was recorded. Then complete-case analysis systematically overstates the result.

Where this shows up next

P10 plots the grouped summary this lesson produced. **P11** turns this pipeline into a script another person can run end to end. **S1** asks what population these exclusions leave you describing, and **S12** returns to the confounding visible in the group summary.

Plotting with Matplotlib

Unit 4 · Lesson 1 of 4 · Python · P10 Reading: 7 pages With practice: ☞ 1.5 hours Assumes: P7

Why this lesson is here

A plot is an argument. It says *look at this, and conclude that* — and it can make a real pattern obvious or an absent one look convincing.

So this lesson is about two things at once: the mechanics of getting a figure out of Matplotlib, and the small number of choices that decide whether the figure tells the truth. Shared scales, honest axes, and not relying on colour alone are not polish. They are the difference between a chart and a claim you can defend.

Before you start

You should be able to:

- build arrays and reduce them along an axis — P7;
- load and group a table — P8, P9.

Quick check. What does `np.linspace(0, 1, 5)` give, and why is it the right tool for plotting a curve?

💡 Show the answer

`[0, 0.25, 0.5, 0.75, 1]` — exactly five points, **including both ends**. That last part is why: `arange` stops before its endpoint, so a curve drawn with it would fall short of the right edge.

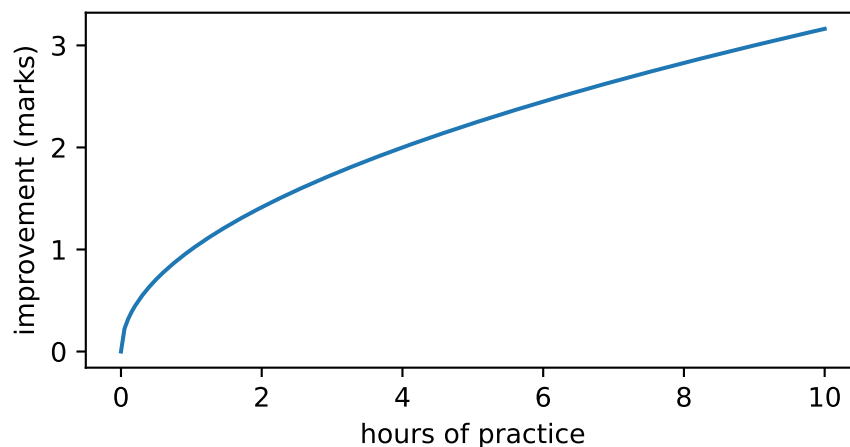
The idea

Matplotlib has two interfaces. Use the one where every command names its target:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 200)

fig, ax = plt.subplots(figsize=(5.2, 2.4))
ax.plot(x, np.sqrt(x))
ax.set(xlabel="hours of practice", ylabel="improvement (marks)")
plt.show()
```



`plt.subplots()` hands back two objects, and knowing which is which removes most confusion:

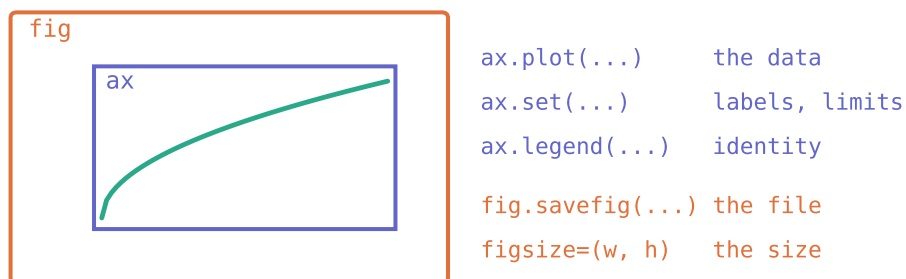


Figure 1: The figure is the page; the axes are a set of coordinates drawn on it. Data goes to the axes; saving and sizing belong to the figure.

Data goes to `ax`. Sizing and saving go to `fig`. Once that split is clear the rest of the library follows.

The mechanics

The four displays that carry most of the work

```
import pandas as pd

cohort = pd.read_csv("data/cohort.csv")
cohort["final_score"] = pd.to_numeric(cohort["final_score"], errors="raise")
data = cohort[cohort["final_score"].between(0, 100) &
             ~ cohort["background"].notna()]

fig, axes = plt.subplots(2, 2, figsize=(6.4, 4.2))

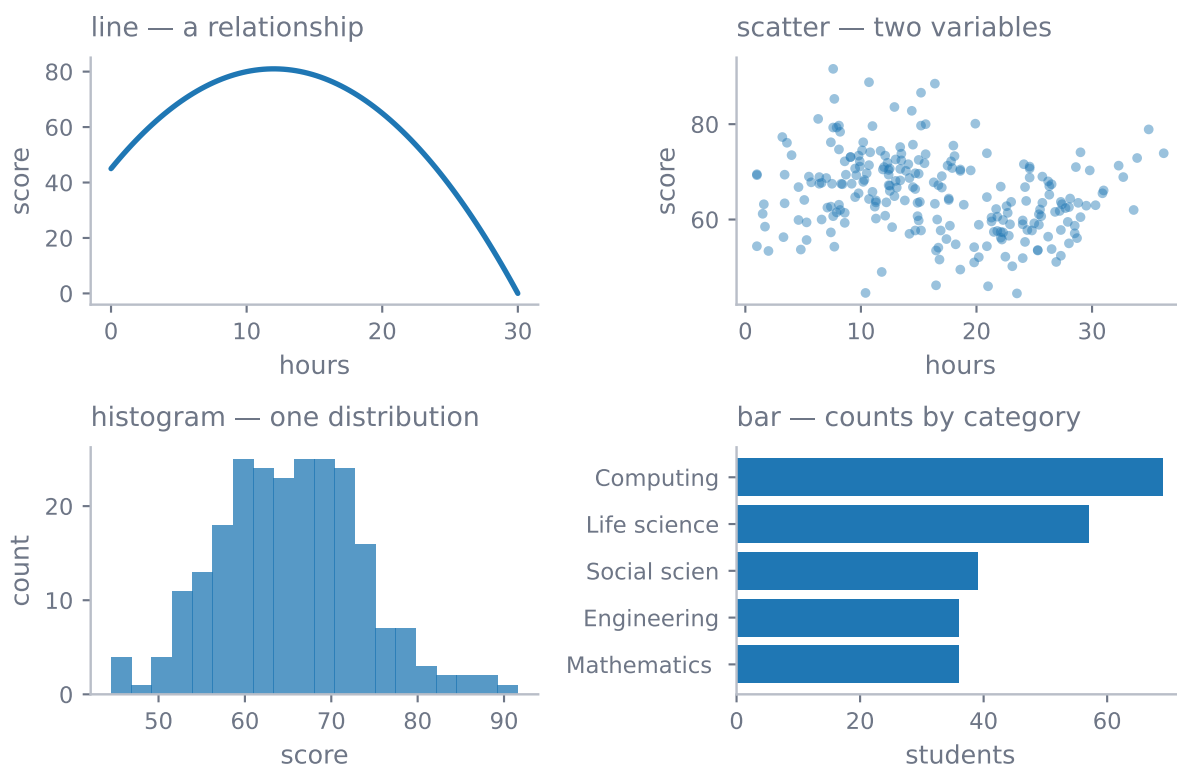
axes[0, 0].plot(np.linspace(0, 30, 100), 45 + 6*np.linspace(0, 30, 100)
               - 0.25*np.linspace(0, 30, 100)**2)
axes[0, 0].set(title="line – a relationship", xlabel="hours", ylabel="score")

axes[0, 1].scatter(data["hours_practice"], data["final_score"], s=7, alpha=0.45)
axes[0, 1].set(title="scatter – two variables", xlabel="hours", ylabel="score")
```

```
# a histogram splits the range into 20 equal intervals (bins) and counts each
axes[1, 0].hist(data["final_score"], bins=20, alpha=0.75)
axes[1, 0].set(title="histogram – one distribution", xlabel="score",
              ylabel="count")

counts = data["background"].value_counts().sort_values()
axes[1, 1].barh(counts.index.str[:12], counts.values)
axes[1, 1].set(title="bar – counts by category", xlabel="students")

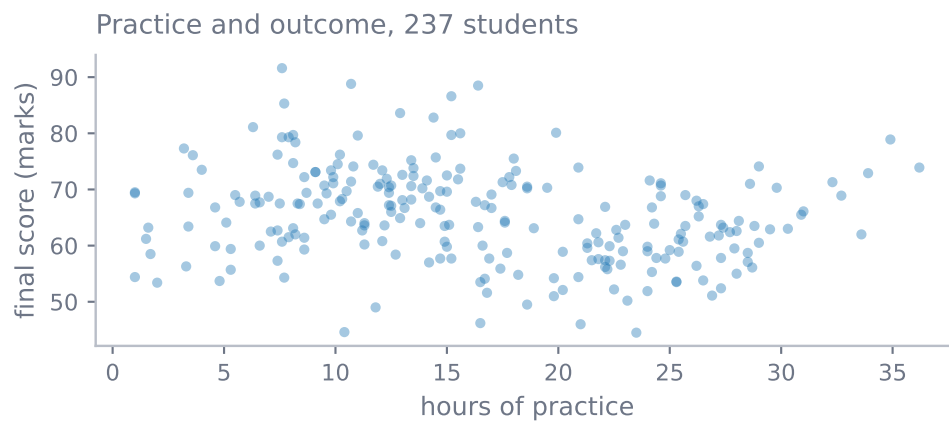
fig.tight_layout()
plt.show()
```



`fig.tight_layout()` stops labels from overlapping their neighbours. Call it before saving, every time.

Labels are not optional

```
fig, ax = plt.subplots(figsize=(5.2, 2.4))
ax.scatter(data["hours_practice"], data["final_score"], s=8, alpha=0.4)
ax.set(xlabel="hours of practice", ylabel="final score (marks)",
      title="Practice and outcome, 237 students")
fig.tight_layout()
plt.show()
```



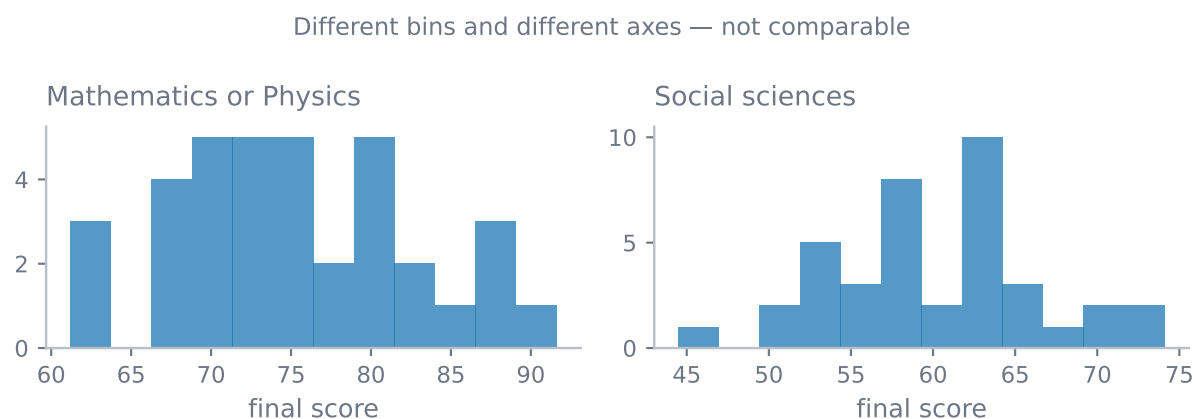
Say what the variable is **and what unit it is in**. “score” is a label; “final score (marks)” is information. A reader who cannot tell whether your y-axis is marks, percentages or z-scores (standardised values, as in P7) cannot check your conclusion.

Comparisons need shared scales

This is the single most consequential honesty choice in the lesson:

```
groups = ["Mathematics or Physics", "Social sciences"]

fig, axes = plt.subplots(1, 2, figsize=(6.4, 2.3))
for ax, name in zip(axes, groups):
    scores = data.loc[data["background"] == name, "final_score"]
    ax.hist(scores, bins=12, alpha=0.75)
    ax.set(title=name[:22], xlabel="final score")
fig.suptitle("Different bins and different axes – not comparable", fontsize=9)
fig.tight_layout()
plt.show()
```

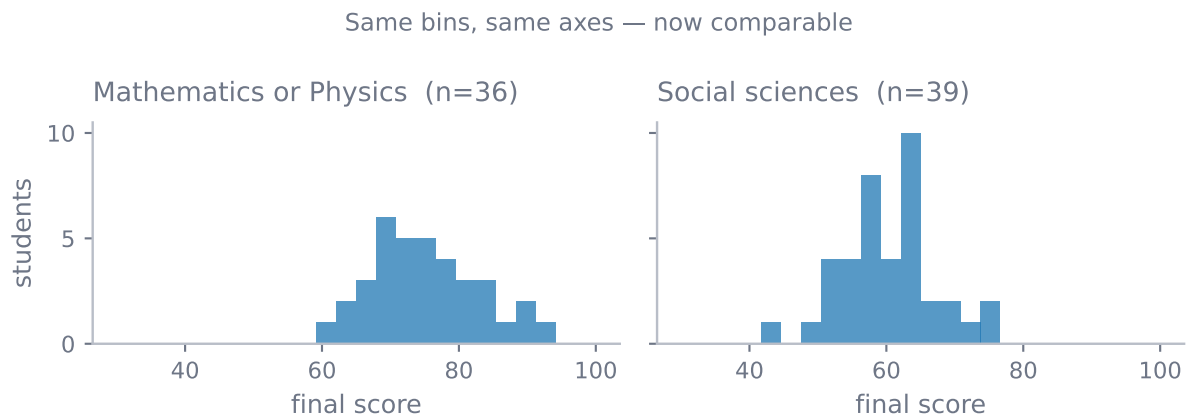


Those two panels look similar. They are not: each chose its own bins and its own axis limits, so the shapes are drawn to different rulers. Fix both:

```
bins = np.linspace(30, 100, 25)

fig, axes = plt.subplots(1, 2, figsize=(6.4, 2.3), sharex=True, sharey=True)
for ax, name in zip(axes, groups):
    scores = data.loc[data["background"] == name, "final_score"]
```

```
ax.hist(scores, bins=bins, alpha=0.75)
ax.set(title=f"{name[:22]} (n={len(scores)})", xlabel="final score")
axes[0].set(ylabel="students")
fig.suptitle("Same bins, same axes – now comparable", fontsize=9)
fig.tight_layout()
plt.show()
```



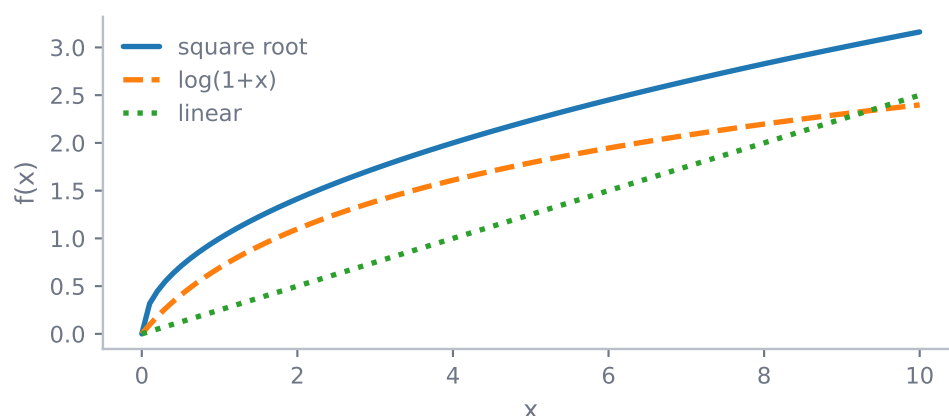
Same bins, sharex, sharey, and the group sizes stated. Now the difference you can see is a difference in the data.

Do not rely on colour alone

Roughly one man in twelve has some form of colour-vision deficiency, and plenty of figures get printed in greyscale anyway. Give every series a second cue — line style, marker shape, or a direct label:

```
x = np.linspace(0, 10, 100)

fig, ax = plt.subplots(figsize=(5.2, 2.4))
ax.plot(x, np.sqrt(x), linestyle="-", label="square root")
ax.plot(x, np.log1p(x), linestyle="--", label="log(1+x)")
ax.plot(x, x/4, linestyle=":", label="linear")
ax.set(xlabel="x", ylabel="f(x)")
ax.legend()
fig.tight_layout()
plt.show()
```



Saving

```
from pathlib import Path

OUT = Path("_outputs"); OUT.mkdir(exist_ok=True)

fig, ax = plt.subplots(figsize=(5.2, 2.4))
ax.hist(data["final_score"], bins=20, alpha=0.8)
ax.set(xlabel="final score (marks)", ylabel="students")
fig.tight_layout()
fig.savefig(OUT / "scores.svg")
plt.close(fig)

print("wrote", (OUT / "scores.svg").stat().st_size, "bytes")
```

wrote 24508 bytes

Use **SVG or PDF** for plots — they are lines and text, and stay sharp at any size. PNG or JPEG are for photographs. And `plt.close(fig)` after saving: a script that makes fifty figures without closing them will eat memory and warn you about it.

Worked example

One figure, made defensible.

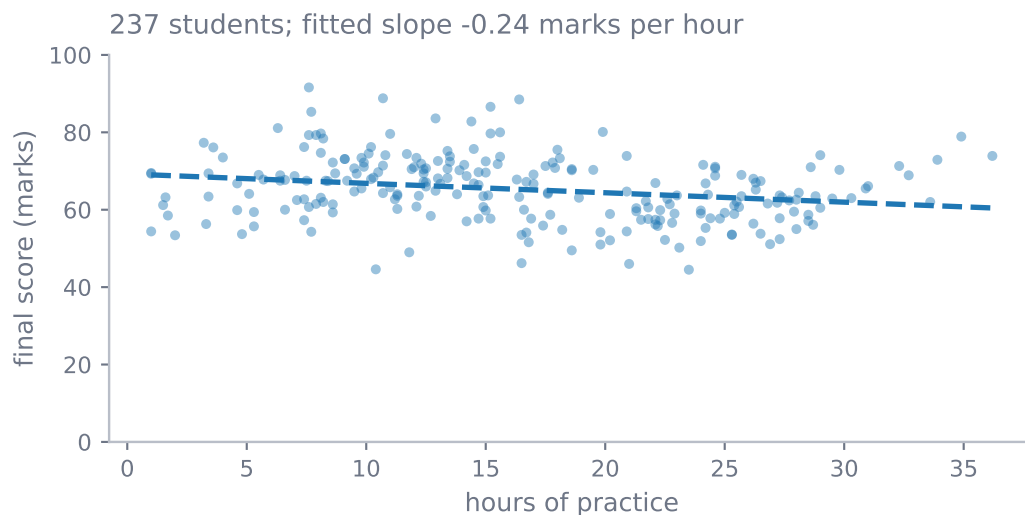
The cohort's overall relationship between practice and score is the dataset's headline oddity. Here it is, plotted carefully:

```
fig, ax = plt.subplots(figsize=(5.6, 2.9))

ax.scatter(data["hours_practice"], data["final_score"],
           s=14, alpha=0.45, edgecolor="none")

# best straight line (degree 1); fit holds [slope, intercept]
fit = np.polyfit(data["hours_practice"], data["final_score"], 1)
line_x = np.array([data["hours_practice"].min(), data["hours_practice"].max()])
# np.polyval(fit, x) gives slope * x + intercept at each x
ax.plot(line_x, np.polyval(fit, line_x), linestyle="--", lw=2)

ax.set(xlabel="hours of practice", ylabel="final score (marks)",
      title=f"237 students; fitted slope {fit[0]:+.2f} marks per hour")
ax.set_ylim(0, 100)
fig.tight_layout()
plt.show()
```



Three decisions in that figure are worth naming.

The y-axis starts at zero and ends at 100, because the score is bounded there. Letting Matplotlib pick would have zoomed into the occupied range and made a modest slope look dramatic. Truncating an axis is the commonest way an honest number becomes a misleading picture.

The slope is in the title, with units. A reader can check the line against the number rather than eyeballing it.

Transparency is set to 0.45, because 237 points overlap. Without it, a dense region reads as one blob and you cannot see where the mass is.

And the thing the figure does *not* say: that practising less causes higher scores. The slope is negative, and it is real — but [S12](#) shows the relationship reverses inside every background group. A plot can be drawn correctly and still invite the wrong conclusion, which is why the caption matters as much as the code.

Try it yourself

Exercise 1 Why is `plt.subplots()` preferable to calling `plt.plot()` directly?

💡 Show the solution

`plt.plot()` draws on whatever figure Matplotlib currently considers active, which is invisible state. In a notebook run out of order, or a script with several figures, “current” is not always what you think.

`fig, ax = plt.subplots()` names both objects, so every subsequent command says what it is acting on. The moment you have two panels it is the only workable option.

Exercise 2 Two histograms are drawn side by side, each with `bins=10` and no shared axes. Give two distinct reasons they cannot be compared.

💡 Show the solution

- **Different bin edges.** `bins=10` means “ten bins spanning *this* data”, so the two panels divide different ranges into different intervals. A bar in one is not the same width as a bar in the other.
- **Different axis limits.** Each panel scales to its own data, so equal bar heights represent

different counts and equal horizontal distances represent different score differences.

Fix both: pass an explicit `bins=` array shared by both, and `sharex=True`, `sharey=True`. Then state each group's `n`, because a taller bar may only mean a bigger group.

Exercise 3 A colleague's bar chart of mean scores has a y-axis running from 58 to 62, and the difference between groups looks enormous. Is the chart wrong? What would you change?

💡 Show the solution

Nothing plotted is false — the bars are at the right heights for that axis. But the *impression* is wrong: a four-mark spread fills the whole panel and reads as a dramatic difference.

For bars, the length is the encoding, so the axis should start at zero; otherwise length no longer means magnitude. If the differences are genuinely small and genuinely interesting, plot the **difference** directly, with an interval showing how precisely it is known — which is S8's subject. The general rule: truncate an axis only when zero is meaningless for the quantity, and say so.

Check it in Python

Plots are easy to get subtly wrong, and three checks catch most of it.

Check the axis limits are what you intended:

```
fig, ax = plt.subplots(figsize=(4.6, 1.9))
ax.hist(data["final_score"], bins=20)
ax.set_ylim(0)
print("x limits:", tuple(round(v, 1) for v in ax.get_xlim()))
print("y limits:", tuple(round(v, 1) for v in ax.get_ylim()))
assert ax.get_ylim()[0] == 0, "count axis must start at zero"
plt.close(fig)
print("count axis starts at zero")
```

```
x limits: (42.1, 94.0)
y limits: (0.0, 26.2)
count axis starts at zero
```

Check nothing was silently dropped. A histogram outside its range simply does not draw the points:

```
scores = data["final_score"]
bins = np.linspace(40, 90, 26) # a range chosen by eye
inside = scores.between(bins[0], bins[-1]).sum()
print(f"{inside} of {len(scores)} points fall inside the plotted range")
assert inside == len(scores), f"{len(scores) - inside} points would be invisible"
```

```
236 of 237 points fall inside the plotted range
```

```
AssertionError: 1 points would be invisible
```

```
-----
AssertionError
```

```
Traceback (most recent call last)
```

```
Cell In[11], line 5
    3 inside = scores.between(bins[0], bins[-1]).sum()
    4 print(f"{inside} of {len(scores)} points fall inside the plotted range")
----> 5 assert inside == len(scores), f"{len(scores) - inside} points would be
      - invisible"
AssertionError: 1 points would be invisible
```

That assertion is not decoration — it just caught a real mistake. A range of 40 to 90 looked reasonable and quietly excluded a student. Derive the range from the data instead of choosing it by eye:

```
bins = np.linspace(scores.min(), scores.max(), 26)
inside = scores.between(bins[0], bins[-1]).sum()
print(f"{inside} of {len(scores)}; range {bins[0]:.1f} to {bins[-1]:.1f}")
assert inside == len(scores)
print("every point is inside the plotted range")
```

```
237 of 237; range 44.5 to 91.6
every point is inside the plotted range
```

Check it survives greyscale. If two series differ only in hue, converting to luminance makes them indistinguishable:

```
fig, ax = plt.subplots(figsize=(4.6, 1.9))
styles = ["-", "--", ":"]
for k, style in enumerate(styles):
    ax.plot(np.linspace(0, 5, 50), np.linspace(0, 5, 50)**(k+1)/5**k,
            linestyle=style, label=f"series {k+1}")
ax.legend()
handles, _ = ax.get_legend_handles_labels()
print("distinct line styles:", len({h.get_linestyle() for h in handles}),
      "of", len(handles), "series")
assert len({h.get_linestyle() for h in handles}) == len(handles)
plt.close(fig)
print("every series has its own line style")
```

```
distinct line styles: 3 of 3 series
every series has its own line style
```

Common wrong turns

Where this usually goes wrong

Truncating the axis on a bar chart The bar's length *is* the number. Starting the axis above zero breaks that and makes small differences look large.

Comparing panels with unshared bins or axes Each panel then uses its own ruler. Pass one explicit bins array and sharex / sharey, and state each group's n.

Relying on colour alone Add a line style, a marker shape, or a direct label. Some readers cannot distinguish your hues, and print often cannot either.

Unlabelled axes, or labels without units "score" could be marks, a percentage or a z-score. The unit belongs in the label.

Using `plt.plot()` in anything with more than one figure It draws on whichever figure is currently active — invisible state that breaks when cells run out of order.

Forgetting `plt.close(fig)` in a script Figures accumulate in memory, and Matplotlib will eventually warn you about it.

Overplotting without transparency Hundreds of opaque points become one shape. `alpha` restores the density.

Check yourself

Answers are at the foot of the page.

1. `fig, ax = plt.subplots()` — which object do you call `savefig` on?
 - a. `ax`
 - b. `fig`
 - c. either
 - d. `plt`
2. Why should a bar chart's value axis start at zero when a scatter plot's need not?
3. Two histograms use `bins=10` each. Name the problem in one sentence.
4. When would PNG be the right format for a figure?
5. A scatter of 5,000 points shows one solid mass. What single argument helps most?

Show the answers

1. **(b) `fig`**. The figure is the page being written out; the axes are drawn on it.
2. Because a bar encodes its value as **length from the baseline**, so moving the baseline breaks the encoding. A scatter encodes value as *position*, and a reader reads the position off the axis — so a zoomed range is legitimate, provided the axis is labelled.
3. `bins=10` means ten bins across *each panel's own range*, so the two panels use different bin widths and are not comparable.
4. For genuinely raster content — a photograph, a satellite image, a rendered heatmap of a very large grid. Line-and-text plots should be SVG or PDF.
5. `alpha` — transparency, so overlapping points accumulate visibly and you can see where the density actually is. (Reducing `s` helps too, and for very large `n` a 2-D histogram may be better than a scatter at all.)

Where this shows up next

P11 saves figures as checked outputs of a script rather than things you made once by hand. **S2** and **S3** choose which of these displays a question calls for. **S12** returns to the scatter in this lesson's worked example and shows why its slope points the wrong way.

Work someone else can rerun

Unit 4 · Lesson 2 of 4 · Python · P11 Reading: 6 pages With practice: ⌚ 2 hours Assumes: P9, P10

Why this lesson is here

An analysis nobody can rerun is an opinion with numbers attached.

The someone who most often cannot rerun it is **you**, four months later, with a marker asking how you got a figure. Everything in this lesson is aimed at that moment: a folder a stranger can open, run one command in, and get your numbers back.

It is also the lesson where the habits of the whole Python track stop being separate tricks and become one way of working.

Before you start

You should be able to:

- load, clean, group and join a table — P9;
- produce and save a labelled figure — P10;
- write a function with an assertion — P5.

Quick check. Why did P9 write the excluded rows to a file rather than just counting them?

💡 Show the answer

Because “we dropped three rows” cannot be checked and “here are the three rows and why” can. Every exclusion is a decision, and a decision nobody can inspect is indistinguishable from a mistake.

The idea

Reproducible does not mean elaborate. It means someone else can get your numbers without asking you anything. That reduces to five questions, and if a folder answers all five it is reproducible enough:

project/	
README.md	the question, and the one command to run
requirements.txt	exact package versions
data/	source data — never edited
analysis.py	loads, cleans, summarises, plots
test_analysis.py	tests of the functions above
outputs/	generated — safe to delete

Figure 1: A layout that answers the five questions. Source data is never edited; everything in outputs/ can be deleted and rebuilt by one command.

The five questions: **what was the question, what data, what code, what environment, and how do I**

run it.

Such a folder is usually kept under **version control** — a tool such as Git that records every saved change to its files — and is then called a **repository**.

The mechanics**Paths that work on someone else's machine**

```
from pathlib import Path

ROOT = Path.cwd()
DATA = ROOT / "data"
print((DATA / "cohort.csv").exists())
```

True

Path joins with / and works identically on Windows, macOS and Linux. C:\Users\me\project\data.csv written as a string works on exactly one machine — yours, until you rename a folder.

In a notebook, Path.cwd() is the folder the notebook lives in, which is why the cell above can use it. A **script** can be run from anywhere, so inside a .py file build paths relative to the script itself, never to wherever the terminal happened to be:

```
ROOT = Path(__file__).resolve().parent # only in a .py file
```

__file__ means “this file”, so it exists only in a script. Typed into a notebook it raises NameError.

Pin the environment

```
import sys, numpy, pandas, matplotlib, scipy
print("python", sys.version.split()[0])
for package in (numpy, pandas, matplotlib, scipy):
    print(f"{package.__name__:<11}", package.__version__)
```

```
python      3.11.14
numpy       1.26.4
pandas      2.3.3
matplotlib  3.10.8
scipy       1.17.0
```

Record those in requirements.txt with ==, not >=:

```
numpy==1.26.4
pandas==2.3.3
matplotlib==3.10.8
scipy==1.17.0
```

>= means “whatever is current when someone installs it”, which is a promise that your results will change one day for reasons unconnected to your work.

Seed anything random, once

```
import numpy as np

rng = np.random.default_rng(2026)
print(rng.normal(size=3).round(4))

again = np.random.default_rng(2026)
print(again.normal(size=3).round(4))
```

```
[-0.7931  0.2406 -1.8963]
[-0.7931  0.2406 -1.8963]
```

Same seed, same numbers, on any machine. Create the generator **once** near the top and pass it where it is needed — creating a fresh one inside a loop restarts the sequence, as P7 showed.

A seed makes a simulation *repeatable*. It does not make it correct, and it does not make a sample representative.

Keep source data immutable

Read from data/, write to outputs/, and never the reverse. If a cleaning step overwrites its own input, the second run operates on different data from the first and you can never get back to the start.

Test the functions, not the script

```
def standardise(values):
    """Return sample-standardised values as a NumPy array."""
    values = np.asarray(values, dtype=float)
    if values.ndim != 1 or values.size < 2:
        raise ValueError("need a one-dimensional sample of at least 2 values")
    sd = values.std(ddof=1)
    if not np.isfinite(sd) or sd == 0:
        raise ValueError("standard deviation must be finite and non-zero")
    return (values - values.mean()) / sd

z = standardise([2, 4, 4, 4, 5, 5, 7, 9])
assert np.isclose(z.mean(), 0)
assert np.isclose(z.std(ddof=1), 1)
print("mean", round(float(z.mean()), 12), " sd", round(float(z.std(ddof=1)), 12))
```

```
mean 0.0 sd 1.0
```

To turn those checks into a test suite, move them out of the notebook into two files next to it. In JupyterLab, **File** ☒ **New** ☒ **Python File** twice:

- analysis.py holds the function — paste the `def standardise` block above, with `import numpy as np` as its first line;

- `test_analysis.py` holds the tests:

```
# test_analysis.py
import numpy as np
import pytest
from analysis import standardise

def test_standardised_has_mean_zero_and_sd_one():
    z = standardise([2, 4, 4, 4, 5, 5, 7, 9])
    assert np.isclose(z.mean(), 0)
    assert np.isclose(z.std(ddof=1), 1)

def test_refuses_constant_input():
    with pytest.raises(ValueError): # passes only if the indented line raises
        ValueError
        standardise([3, 3, 3])
```

pytest is not one of the packages from setup. In JupyterLab's terminal (**File** ▢ **New** ▢ **Terminal**), install it once and run the tests:

```
python -m pip install pytest==8.3.4
python -m pytest
```

pytest finds tests by name: files called `test_*.py`, functions called `test_*`. That is why the file is `test_analysis.py` — call it checks `.py` and pytest reports that no tests ran. Two passing tests print as 2 passed.

A test that a function *refuses* bad input is worth as much as one that it handles good input — often more, because that is the path nobody exercises by hand.

i A passing test says the behaviour matched your expectation. It does not say your expectation was right. Tests catch **regressions** (something that used to work, broken by a later change) and typos; they cannot tell you that you standardised the wrong column.

Notebooks: restart and run all

A notebook shows the *output* of cells, not the order they ran in. A cell you edited and never re-ran leaves a stale number on screen that no longer follows from the code above it.

Before you submit or share anything: **Restart Kernel and Run All Cells**, and check it still works from top to bottom. If it does not, it was never going to work for anyone else either.

When logic becomes reusable, move it into a `.py` file and import it. Keep the notebook for narrative.

Worked example

The pipeline as a script someone else can run.

```

from pathlib import Path
import pandas as pd

ROOT = Path.cwd()          # in a notebook; saved as analysis.py, use
↳ Path(__file__).resolve().parent
DATA, OUT = ROOT / "data", ROOT / "_outputs"
OUT.mkdir(exist_ok=True)
SEED = 2026

def load(path):
    """Load the cohort file and convert types deliberately."""
    frame = pd.read_csv(path)
    for column in ("diagnostic_score", "hours_practice", "final_score"):
        frame[column] = pd.to_numeric(frame[column], errors="raise")
    return frame

def split(frame):
    """Separate analysable rows from excluded ones."""
    ok = (frame["final_score"].between(0, 100)
          & frame["final_score"].notna()
          & frame["background"].notna())
    kept, dropped = frame.loc[ok].copy(), frame.loc[~ok].copy()
    assert len(kept) + len(dropped) == len(frame)
    return kept, dropped

def summarise(frame):
    """One row per background, with counts and spread."""
    return (frame.groupby("background", as_index=False)
            .agg(n=("final_score", "size"),
                 mean_score=("final_score", "mean"),
                 sd_score=("final_score", "std"))
            .round(2))

cohort = load(DATA / "cohort.csv")
analysis, excluded = split(cohort)
summary = summarise(analysis)

summary.to_csv(OUT / "group_summary.csv", index=False)
excluded.to_csv(OUT / "excluded_rows.csv", index=False)

assert summary["n"].sum() == len(analysis)
print(f"{len(cohort)} rows in, {len(analysis)} analysed, {len(excluded)}
↳ excluded")
print(f"wrote {len(list(OUT.glob('*.csv')))} files to {OUT.name}/")
summary

```

240 rows in, 237 analysed, 3 excluded

wrote 2 files to `_outputs/`

	background	n	mean_score	sd_score
0	Computing	69	65.85	6.60
1	Engineering	36	67.15	5.33
2	Life sciences	57	60.51	7.29
3	Mathematics or Physics	36	75.28	7.63
4	Social sciences	39	60.21	6.58

Four named functions, each testable on its own; every path built from `ROOT`; every exclusion written out; two assertions guarding the arithmetic. Nothing here is clever, and that is rather the point.

The README that goes with it

```
# Cohort outcomes by prior background

**Question.** Does prior background relate to final score, and how do the
groups differ in practice hours?

**Data.** `data/cohort.csv` – 240 synthetic records. Generated by
`data/make_cohort.py` with seed 2026. Not real students.

**Setup.**
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

**Run.**
python analysis.py

**Outputs.** `_outputs/group_summary.csv`, `_outputs/excluded_rows.csv`.
Three rows are excluded: one missing score, one missing background, one score
of 187 (impossible). See `_outputs/excluded_rows.csv`.
```

Ten lines. It answers all five questions, and it is the difference between a folder and a piece of work.

Try it yourself

Exercise 1 A notebook produces a figure you cannot reproduce after restarting it. Name two likely causes.

Show the solution

- **Cells were run out of order.** A name still held a value from a cell that has since been edited or deleted, so the figure depended on state not visible in the notebook as written.
- **An unseeded random step.** Anything using `np.random` without a fixed seed gives different numbers each run.

A third, less often noticed: the notebook read a file that has since been overwritten by a later cell

— the immutable-source-data rule exists for exactly this.

Exercise 2 Why is `pandas>=2.0` in a requirements file worse than `pandas==2.3.3`?

 Show the solution

`>=` installs whatever is current on the day someone runs it. A default changes, a function is deprecated, and your numbers move — for reasons unconnected to your analysis, with nothing in your project recording what changed.

`==` fixes it. When you *choose* to upgrade, you change one line, rerun the tests, and see what moved. That is a decision with evidence rather than a surprise.

Exercise 3 Your analysis reads `data/cohort.csv`, cleans it, and writes the cleaned version back to the same path. Give two things that go wrong.

 Show the solution

- **The second run cleans already-cleaned data.** If cleaning is not exactly idempotent (giving the same result when run twice as when run once) — dropping rows, rescaling, recoding — run two gives a different answer from run one, and neither is reproducible.
- **The original is gone.** You cannot check what the raw values were, cannot rerun with a different rule, and cannot show anyone what you excluded.

Write to `outputs/`. Treat `data/` as read-only, and if you can, make it literally read-only so the mistake becomes an error.

Check it in Python

Record the environment inside the output, so a result carries its own provenance:

```
import sys, json, numpy, pandas
provenance = {
    "python": sys.version.split()[0],
    "numpy": numpy.__version__,
    "pandas": pandas.__version__,
    "seed": SEED,
    "rows_in": int(len(cohort)),
    "rows_analysed": int(len(analysis)),
    "rows_excluded": int(len(excluded)),
}
(OUT / "provenance.json").write_text(json.dumps(provenance, indent=2))
print(json.dumps(provenance, indent=2))
```

```
{
  "python": "3.11.14",
  "numpy": "1.26.4",
  "pandas": "2.3.3",
  "seed": 2026,
  "rows_in": 240,
  "rows_analysed": 237,
```

```
"rows_excluded": 3
}
```

Six months on, that file answers “which versions produced this?” without anyone having to remember.

Assert the invariants that must hold, so a broken run stops rather than producing a plausible wrong answer:

```
assert len(analysis) + len(excluded) == len(cohort), "rows lost in the split"
assert summary["n"].sum() == len(analysis), "group sizes do not match"
assert analysis["final_score"].between(0, 100).all(), "impossible score survived"
assert not analysis["background"].isna().any(), "missing background survived"
print("all four invariants hold")
```

```
all four invariants hold
```

Confirm the outputs are actually there, and that no new file appeared in the source directory. A listing cannot show that an existing file was overwritten; making data/ read-only, as in Exercise 3, is what prevents that:

```
for name in ("group_summary.csv", "excluded_rows.csv", "provenance.json"):
    path = OUT / name
    print(f" {name:<22} {path.stat().st_size:>6} bytes")
print("\nsource data untouched:", sorted(p.name for p in (ROOT /
    ↳ 'data').glob('*.*csv')))
```

```
group_summary.csv          177 bytes
excluded_rows.csv          220 bytes
provenance.json            147 bytes
```

```
source data untouched: ['background_meta.csv', 'cohort.csv']
```

Getting help, and using an assistant

You are working alone, so how you ask for help matters.

A good request contains four things: **a minimal example** that reproduces the problem, **what you expected**, **what happened** including the full final line of the traceback, and **your versions** from the check above. Assembling that often solves the problem before you send it — which is itself a reason to do it.

On AI assistants, plainly. They are useful and you will use them. Two things worth deciding now, while nothing is at stake.

Use one as a tutor, not an answer key. “Explain why this raises” and “what does `ddof` change here” leave you able to do the next one. “Write this for me” does not, and the gap shows up in the first thing you have to do unaided.

Never paste anything you would not email to a stranger — unpublished data, personal records, credentials, assessed work belonging to someone else.

And when you do keep generated code: read every line, and be able to say what it does. Code you cannot explain is code you cannot debug, defend, or fix at eleven at night before a deadline. The academic-integrity line at your university will be written down somewhere; find it and read it before you need it, not after.

Common wrong turns

Where this usually goes wrong

Absolute paths `/Users/me/project/data.csv` works on exactly one machine. Build from `Path(__file__).resolve().parent`.

Writing cleaned data over the source The second run then cleans already-cleaned data, and the original is gone. Source data is read-only; generated files go in `outputs/`.

>= in requirements Your results change when an unrelated package updates. Pin with `==` and upgrade deliberately.

Seeding inside a loop Restarts the sequence every iteration, so every draw is identical. Create the generator once.

Trusting notebook output The visible numbers may come from cells that no longer exist. Restart and run all before believing anything.

Treating a passing test as proof of correctness It shows the behaviour matched your expectation. Your expectation can be wrong, and no test will tell you.

Pasting code you cannot explain It works until it does not, and then you cannot fix it. Read every line you keep.

Check yourself

Answers are at the foot of the page.

1. Which belongs in version control?
 - a. `outputs/figure.svg`
 - b. `requirements.txt`
 - c. `.venv/`
 - d. all three
2. Why build paths with `pathlib` rather than string concatenation?
3. A colleague's notebook gives different numbers each run. Two likely causes?
4. Your tests pass but a reviewer finds the analysis answers the wrong question. What does that say about tests?
5. Name two things that belong in a README and one thing that does not belong in the repository at all.

Show the answers

1. **(b) requirements.txt.** It is source: small, hand-written, and needed to rebuild everything else. `outputs/` is generated and should be reproducible from the code; `.venv/` is large, machine-specific, and rebuilt from `requirements.txt`.

2. Because separators differ between operating systems, and `pathlib` joins correctly on all of them. It also makes relative-to-the-script paths natural, which is what stops a project from working on one machine only.
3. An unseeded random step, or cells run out of order so a value came from code that no longer exists. A third: reading a file that a later cell overwrites.
4. That tests check **internal consistency**, not scientific validity. They confirm the code does what you told it to; whether that was the right thing to tell it is a question no test can answer.
5. In the README: the question, the data and where it came from, setup steps, the exact run command, what outputs appear, and known limitations. Not in the repository at all: credentials, API keys, personal data, or anyone else's assessed work.

Where this shows up next

That completes the Python track. Everything from here uses it.

M1 starts the Mathematics track, and every lesson in it ends with a *Check it in Python* block built on these habits. **S1** opens Statistics by asking what question an analysis is answering — the same question this lesson's README begins with.